

C++ data structures quick revision guide with syntax

01/15/2026 22:16:17

Create a C++ Data Structures quick revision guide with syntax and...

C++ Data Structures Quick Revision Guide 1. Arrays & Vectors Static Arr...

Create a C++ Data Structures REVISION + QUICK UNDERSTANDING guid...

 C++ DSA Quick Revision Guide (Interview-Focused) **1** Arrays &...

Yes but before that guide one by one proper and deep

 C++ DSA Deep Dive Guide (One by One) **1** ARRAYS (Static Array) ...

Continue

2 VECTOR (Dynamic Array)  What is Vector? Vector ek dynamic array...

Continue

3 STRINGS  What is String? String ek sequence of characters hai jo te...

Continue

3 STRINGS (Continued) Problem 5: First Non-Repeating Character char...

Yes

4 LINKED LIST  What is Linked List? Linked List ek linear data structur...

Yes proepr

4 LINKED LIST (Continued)  APPROACH 2: STL list<> (Doubly Linked...

Yes continue

5 STACK  What is Stack? Stack ek LIFO (Last In First Out) data structu...

Continue deep proper

6 QUEUE & DEQUE  PART A: QUEUE  What is Queue? Queue ek FIF...

Not need continue next topic

7 HASH MAP (unordered_map)  What is Hash Map? Hash Map...

-  Create a C++ Data Structures quick revision guide with syntax and operations for DSA problem-solving.**

For each data structure, show:

1. **Declaration/Creation** - how to initialize
2. **Insertion** - add elements (all methods)
3. **Deletion** - remove elements (all methods)
4. **Access** - read/retrieve elements
5. **Traversal** - iterate through elements
6. **Search** - find elements
7. **Update** - modify elements
8. **Size/Empty checks**
9. **Common operations** - unique to that structure
10. **Time complexity** for each operation

Cover these data structures:

Arrays & Vectors:

- Static array, dynamic array (vector)
- Operations: create, insert, delete, access, resize, sort, reverse

Strings:

- Creation, concatenation, substring, find, replace, conversion, parsing

Linked List:

- Singly/doubly linked list (or using `list` STL)
- Insert at beginning/end/middle, delete, traverse, reverse

Stack:

- Push, pop, top, empty check

Queue & Deque:

- Enqueue, dequeue, front, back operations

HashMap/Unordered_map:

- Insert, delete, search, iterate, count, check existence

Set/Unordered_set:

- Insert, erase, find, count, iterate

Map (ordered):

- Insert, delete, access by key, iterate, lower_bound, upper_bound

Priority Queue (Heap):

- Push, pop, top (max-heap and min-heap)

Pair & Tuple:

- Creation, access elements

Format each as:

```
// Declaration
code here

// Operation 1
code here

// Operation 2
code here
```

Keep it concise - focus on practical syntax I need while coding, not theory." - start first and make good with simple explain also

C++ Data Structures Quick Revision Guide

1. Arrays & Vectors

Static Array

```
// Declaration
int arr[5]; // uninitialized
int arr[5] = {1, 2, 3, 4, 5}; // initialized
int arr[] = {1, 2, 3}; // size auto-calculated

// Access - O(1)
int x = arr[0]; // first element
arr[2] = 10; // modify element

// Size
int size = sizeof(arr)/sizeof(arr[0]);

// Traversal - O(n)
for(int i = 0; i < size; i++) {
    cout << arr[i] << " ";
}

// Range-based loop
for(int x : arr) {
    cout << x << " ";
}

// Sort - O(n log n)
sort(arr, arr + size); // ascending
sort(arr, arr + size, greater<int>()); // descending
```

```

// Reverse - O(n)
reverse(arr, arr + size);

// Search - O(n) linear, O(log n) binary (if sorted)
int* ptr = find(arr, arr + size, 3); // returns pointer
if(ptr != arr + size) cout << "Found";

// Binary search (sorted array) - O(log n)
bool found = binary_search(arr, arr + size, 3);

```

Dynamic Array (Vector)

```

#include <vector>

// Declaration
vector<int> v; // empty vector
vector<int> v(5); // size 5, default values
vector<int> v(5, 10); // size 5, all elements = 10
vector<int> v = {1, 2, 3, 4, 5}; // initialized
vector<int> v(arr, arr + size); // from array

// Insertion
v.push_back(6); // O(1) - add at end
v.insert(v.begin() + 2, 99); // O(n) - insert at index 2
v.insert(v.end(), 100); // O(1) - insert at end
v.emplace_back(7); // O(1) - construct in place

// Deletion
v.pop_back(); // O(1) - remove last
v.erase(v.begin() + 2); // O(n) - erase at index 2
v.erase(v.begin(), v.begin() + 3); // O(n) - erase range [0,3)
v.clear(); // O(n) - remove all

```

```
// Access
```

```
int x = v[0];
```

```
// O(1) - no bounds check
```

```
int y = v.at(0);
```

```
// O(1) - with bounds check
```

```
int first
```

07/01/2026, 13:22:44



👤 Create a C++ Data Structures REVISION + QUICK UNDERSTANDING guide for DSA problem-solving. I want short explanation + working example + syntax, not long theory. 🧠 TEACHING RULES (VERY IMPORTANT) Use simple Indian English Explanation must be max 2-3 lines Show how it works using a small example Show actual C++ STL syntax Keep it concise, practical, and interview-focused No deep internals, no long paragraphs 📌 FOR EACH DATA STRUCTURE, SHOW EXACTLY THIS: 1 What it is (1-2 lines) 2 How it works (small example) 3 Common operations with syntax: Declaration / Initialization Insertion Deletion Access Traversal Search Update Size / Empty check Special operations (if any) 4 Time Complexity (one simple table or list) 📖 DATA STRUCTURES TO COVER (DO NOT MISS) ♦ Arrays & Vectors Static array Vector insert, delete, access, resize, sort, reverse ♦ Strings Creation Concatenation Substring Find Replace Conversion (string ↔ int) ♦ Linked List list Insert (front, back) Delete Traverse Reverse ♦ Stack push, pop, top, empty ♦ Queue & Deque enqueue, dequeue front, back ♦ Hash Map unordered_map insert, erase, find, count existence check ♦ Set set unordered_set insert, erase, find ♦ Map (Ordered) insert, erase access by key lower_bound, upper_bound ♦ Priority Queue (Heap) max heap min heap push, pop, top ♦ Pair & Tuple creation access elements

07/01/2026, 13:23:24

C++ DSA Quick Revision Guide (Interview-Focused)

1 Arrays & Vectors

📌 Static Array

What: Fixed size array, size cannot change after declaration.

Example: `int arr[5] = {10, 20, 30, 40, 50}` creates array with 5 elements.

```

// Declaration
int arr[5] = {10, 20, 30, 40, 50};
int arr2[100]; // uninitialized

// Access - O(1)
cout << arr[0]; // 10
arr[2] = 99;     // update

// Size
int n = sizeof(arr)/sizeof(arr[0]); // 5

// Traversal - O(n)
for(int i = 0; i < n; i++)
    cout << arr[i] << " ";

// Sort - O(n log n)
sort(arr, arr + n); // ascending
sort(arr, arr + n, greater<int>()); // descending

// Reverse - O(n)
reverse(arr, arr + n);

// Search - O(n)
int* found = find(arr, arr + n, 30);
if(found != arr + n) cout << "Found at index " << (found - arr);

// Binary Search (sorted) - O(log n)
bool exists = binary_search(arr, arr + n, 30);

```

Time Complexity:

- Access: $O(1)$
- Search: $O(n)$ linear, $O(\log n)$ binary
- Sort: $O(n \log n)$

Vector (Dynamic Array)

What: Dynamic array jo automatically grow/shrink hota hai.

Example: `vector<int> v = {10, 20, 30}` creates resizable array.

```
#include <vector>

// Declaration
vector<int> v; // empty
vector<int> v = {10, 20, 30}; // initialized
vector<int> v(5, 100); // size 5, all = 100

// Insertion
v.push_back(40); // O(1) - end me add
v.insert(v.begin() + 1, 15); // O(n) - index 1 pe insert
v.emplace_back(50); // O(1) - faster than push_back

// Deletion
v.pop_back(); // O(1) - last element remove
v.erase(v.begin() + 2); // O(n) - index 2 remove
v.clear(); // O(n) - sab remove

// Access - O(1)
cout << v[0]; // 10
cout << v.at(1); // bounds check ke sath
cout << v.front(); // first element
cout << v.back(); // last element

// Update - O(1)
v[0] = 99;

// Size/Empty
int size = v.size();
```

```
bool empty = v.empty();

// Traversal
for(int i = 0; i < v.size(); i++)
    cout << v[i] << " ";

for(int x : v) // range-based
    cout << x << " ";

// Sort/Reverse
sort(v.begin(), v.end());
reverse(v.begin(), v.end());

// Search
auto it = find(v.begin(), v.end(), 30);
if(it != v.end()) cout << "Found";
```

Time Complexity:

- Access: $O(1)$
- Insert/Delete at end: $O(1)$ amortized
- Insert/Delete at middle: $O(n)$
- Search: $O(n)$

2 Strings

What: Character array jo text store karta hai.

Example: `string s = "hello"` creates string with 5 characters.

```
#include <string>
```

```

// Declaration
string s = "hello";
string s2("world");
string s3(5, 'a'); // "aaaaa"

// Concatenation
string s = "hello" + " " + "world"; // "hello world"
s += "!"; // "hello world!"
s.append(" bye"); // "hello world! bye"

// Access - O(1)
cout << s[0]; // 'h'
cout << s.at(1); // 'e'
s[0] = 'H'; // update

// Size
int len = s.size(); // or s.length()
bool empty = s.empty();

// Substring - O(n)
string sub = s.substr(0, 5); // "hello" (start, length)
string sub2 = s.substr(6); // "world!" (start to end)

// Find - O(n*m)
int pos = s.find("world"); // 6 (index)
if(pos != string::npos)
    cout << "Found at " << pos;

int pos2 = s.find('o'); // first 'o' at index 4
int pos3 = s.rfind('o'); // last 'o' from right

// Replace - O(n)
s.replace(0, 5, "hi"); // replace "hello" with "hi"

// Insert/Erase

```

```

s.insert(2, "XX");           // insert at index 2
s.erase(0, 2);              // erase 2 chars from index 0

// Compare
if(s1 == s2) cout << "Equal";
if(s1.compare(s2) == 0) cout << "Equal";

// Conversion
int num = stoi("123");       // string to int
long l = stol("12345");
double d = stod("3.14");
string s = to_string(123);   // int to string

// Character checks
if(isdigit(s[0])) cout << "digit";
if(isalpha(s[0])) cout << "alphabet";

// Case conversion
transform(s.begin(), s.end(), s.begin(), ::toupper); // HELLO
transform(s.begin(), s.end(), s.begin(), ::tolower); // hello

// Reverse
reverse(s.begin(), s.end());

// Sort
sort(s.begin(), s.end());

// Traversal
for(char c : s) cout << c;

```

Time Complexity:

- Access: $O(1)$
- Find: $O(n \times m)$
- Substring: $O(n)$

- Concatenation: $O(n)$

3 Linked List

What: Nodes ka collection, har node me data + next pointer hota hai.

Example: `list<int> l = {10, 20, 30}` creates doubly linked list.

```
#include <list>

// Declaration
list<int> l;
list<int> l = {10, 20, 30, 40};

// Insertion
l.push_back(50);      //  $O(1)$  - end me
l.push_front(5);      //  $O(1)$  - start me
l.insert(next(l.begin(), 2), 25); //  $O(n)$  - position pe

// Deletion
l.pop_back();         //  $O(1)$  - last remove
l.pop_front();        //  $O(1)$  - first remove
l.erase(next(l.begin(), 1)); //  $O(n)$  - position se remove
l.remove(30);         //  $O(n)$  - value se remove

// Access
int first = l.front(); //  $O(1)$ 
int last = l.back();   //  $O(1)$ 
// Note: l[i] not possible in list

// Size/Empty
int size = l.size();
bool empty = l.empty();
```

```
// Traversal - O(n)
for(auto it = l.begin(); it != l.end(); it++)
    cout << *it << " ";

for(int x : l)
    cout << x << " ";

// Reverse - O(n)
l.reverse();

// Sort - O(n log n)
l.sort();
l.sort(greater<int>());

// Search - O(n)
auto it = find(l.begin(), l.end(), 30);
if(it != l.end()) cout << "Found";

// Clear
l.clear();
```

Time Complexity:

- Insert/Delete at front/back: $O(1)$
- Insert/Delete at middle: $O(n)$
- Access: $O(n)$
- Search: $O(n)$

4 Stack

What: LIFO (Last In First Out) structure, last me jo add hua wo pehle nikalega.

Example: Books ka pile - upar wali book pehle nikalti hai.

```
#include <stack>

// Declaration
stack<int> st;

// Insertion (Push) - O(1)
st.push(10);
st.push(20);
st.push(30);

// Access Top - O(1)
cout << st.top(); // 30 (last inserted)

// Deletion (Pop) - O(1)
st.pop();          // removes 30
cout << st.top();  // 20

// Size/Empty
int size = st.size();
bool empty = st.empty();

// Traversal (stack ko traverse karne ke liye pop karna padega)
while(!st.empty()) {
    cout << st.top() << " ";
    st.pop();
}
```

Time Complexity:

- Push: O(1)
- Pop: O(1)
- Top: O(1)

Use Cases: Function calls, undo operations, expression evaluation, DFS

5 Queue

What: FIFO (First In First Out) structure, jo pehle aaya wo pehle nikalega.

Example: Movie ticket counter - jo pehle line me aaya wo pehle ticket lega.

```
#include <queue>

// Declaration
queue<int> q;

// Insertion (Enqueue) - O(1)
q.push(10);
q.push(20);
q.push(30);

// Access
cout << q.front(); // O(1) - 10 (first inserted)
cout << q.back();  // O(1) - 30 (last inserted)

// Deletion (Dequeue) - O(1)
q.pop();           // removes 10
cout << q.front(); // 20

// Size/Empty
int size = q.size();
bool empty = q.empty();

// Traversal (queue ko empty karna padega)
while(!q.empty()) {
    cout << q.front() << " ";
```



```
q.pop();  
}
```

Time Complexity:

- Push: $O(1)$
- Pop: $O(1)$
- Front/Back: $O(1)$

Use Cases: BFS, task scheduling, printer queue

Deque (Double Ended Queue)

What: Dono end se insert/delete kar sakte ho.

Example: `deque<int> dq = {10, 20, 30}` allows operations at both ends.

```
#include <deque>  
  
// Declaration  
deque<int> dq;  
deque<int> dq = {10, 20, 30};  
  
// Insertion  
dq.push_back(40);    //  $O(1)$  - end me  
dq.push_front(5);    //  $O(1)$  - start me  
dq.insert(dq.begin() + 2, 15); //  $O(n)$  - middle me  
  
// Deletion  
dq.pop_back();        //  $O(1)$   
dq.pop_front();       //  $O(1)$   
dq.erase(dq.begin() + 1); //  $O(n)$ 
```

```

// Access - O(1)
cout << dq[0];
cout << dq.at(1);
cout << dq.front();
cout << dq.back();

// Size/Empty
int size = dq.size();
bool empty = dq.empty();

// Traversal
for(int x : dq)
    cout << x << " ";

```

Time Complexity:

- Insert/Delete at both ends: $O(1)$
- Access: $O(1)$
- Insert/Delete at middle: $O(n)$

6 Hash Map (Unordered Map)

What: Key-Value pairs store karta hai, unordered rehta hai, fast access.

Example: `unordered_map<string, int> m = {{"apple", 5}}` stores fruit names with counts.

```

#include <unordered_map>

// Declaration
unordered_map<int, int> m;
unordered_map<string, int> m = {{"apple", 5}, {"banana", 3}};

```

```

// Insertion - O(1) average
m[1] = 100;           // key=1, value=100
m[2] = 200;
m.insert({3, 300});
m.insert(make_pair(4, 400));

// Access - O(1) average
cout << m[1];        // 100
cout << m.at(2);     // 200 (throws error if not found)

// Update - O(1) average
m[1] = 999;

// Check existence - O(1) average
if(m.find(1) != m.end())
    cout << "Key 1 exists";

if(m.count(1))
    cout << "Key 1 exists";

// Deletion - O(1) average
m.erase(1);          // remove key 1

// Size/Empty
int size = m.size();
bool empty = m.empty();

// Traversal - O(n)
for(auto it : m)
    cout << it.first << " -> " << it.second << "\n";

for(auto [key, val] : m) // C++17
    cout << key << " -> " << val << "\n";

```

```
// Clear  
m.clear();
```

Time Complexity:

- Insert: $O(1)$ average, $O(n)$ worst
- Access: $O(1)$ average, $O(n)$ worst
- Delete: $O(1)$ average
- Search: $O(1)$ average

Use Cases: Frequency counting, caching, two sum problem

7 Set (Unordered Set)

What: Unique elements store karta hai, duplicates nahi aate, unordered.

Example: `unordered_set<int> s = {1, 2, 3}` stores unique numbers.

```
#include <unordered_set>  
  
// Declaration  
unordered_set<int> s;  
unordered_set<int> s = {10, 20, 30};  
  
// Insertion -  $O(1)$  average  
s.insert(40);  
s.insert(10); // duplicate, not added  
  
// Check existence -  $O(1)$  average  
if(s.find(20) != s.end())  
    cout << "20 exists";
```

```

if(s.count(20))
    cout << "20 exists";

// Deletion - O(1) average
s.erase(20);

// Size/Empty
int size = s.size();
bool empty = s.empty();

// Traversal - O(n) (no particular order)
for(int x : s)
    cout << x << " ";

// Clear
s.clear();

```

Time Complexity:

- Insert: O(1) average
- Find: O(1) average
- Delete: O(1) average

Set (Ordered)

What: Unique elements, sorted order me store hote hai.

Example: `set<int> s = {30, 10, 20}` automatically becomes {10, 20, 30}.

```

#include <set>

// Declaration
set<int> s;

```

```

set<int> s = {30, 10, 20}; // stored as {10, 20, 30}

// Insertion - O(log n)
s.insert(40);

// Find - O(log n)
if(s.find(20) != s.end())
    cout << "Found";

// Deletion - O(log n)
s.erase(20);

// Traversal - O(n) (sorted order)
for(int x : s)
    cout << x << " "; // 10 20 30 40

// Lower/Upper bound - O(log n)
auto it = s.lower_bound(25); // >= 25 ka first element
auto it2 = s.upper_bound(25); // > 25 ka first element

// Size/Empty
int size = s.size();
bool empty = s.empty();

```

Time Complexity:

- Insert/Delete/Find: $O(\log n)$

Use Cases: Remove duplicates + sorted order

8 Map (Ordered)

What: Key-Value pairs, sorted by key, ordered rehta hai.

Example: `map<int, string> m = {{1, "one"}}` stores numbers to words mapping.

```
#include <map>

// Declaration
map<int, string> m;
map<int, string> m = {{3, "three"}, {1, "one"}, {2, "two"}};
// stored as {1:"one", 2:"two", 3:"three"}

// Insertion - O(log n)
m[4] = "four";
m.insert({5, "five"});

// Access - O(log n)
cout << m[1];           // "one"
cout << m.at(2);        // "two"

// Update - O(log n)
m[1] = "ONE";

// Find - O(log n)
if(m.find(2) != m.end())
    cout << "Key 2 exists";

if(m.count(2))
    cout << "Key 2 exists";

// Deletion - O(log n)
m.erase(2);

// Size/Empty
int size = m.size();
bool empty = m.empty();

// Traversal - O(n) (sorted by key)
```

```

for(auto [key, val] : m)
    cout << key << " -> " << val << "\n";

// Lower/Upper bound -  $O(\log n)$ 
auto it = m.lower_bound(3); // key >= 3
auto it2 = m.upper_bound(3); // key > 3

// Clear
m.clear();

```

Time Complexity:

- Insert/Access/Delete: $O(\log n)$
- Lower/Upper bound: $O(\log n)$

Difference from unordered_map: Map is sorted, operations $O(\log n)$.
 Unordered_map is unsorted, operations $O(1)$ average.

9 Priority Queue (Heap)

What: Max/Min element ko hamesha top pe rakhta hai.

Example: `priority_queue<int> pq` max element ko top pe rakhega.

Max Heap (Default)

```

#include <queue>

// Declaration - Max Heap
priority_queue<int> pq; // max element on top

// Insertion -  $O(\log n)$ 
pq.push(10);

```



```

pq.push(30);
pq.push(20);

// Access Top - O(1)
cout << pq.top(); // 30 (max element)

// Deletion - O(log n)
pq.pop();          // removes 30
cout << pq.top();  // 20

// Size/Empty
int size = pq.size();
bool empty = pq.empty();

// Traversal (pop karke)
while(!pq.empty()) {
    cout << pq.top() << " ";
    pq.pop();
} // prints: 20 10

```

Min Heap

```

// Declaration - Min Heap
priority_queue<int, vector<int>, greater<int>> pq; // min on top

// Insertion - O(log n)
pq.push(10);
pq.push(30);
pq.push(20);

// Access Top - O(1)
cout << pq.top(); // 10 (min element)

// Deletion - O(log n)

```

```
pq.pop();           // removes 10
cout << pq.top();   // 20
```

Custom Comparator (for pairs/objects)

```
// Max heap for pairs based on second element
priority_queue<pair<int,int>> pq;
pq.push({1, 100});
pq.push({2, 50});
cout << pq.top().second; // 100

// Min heap for pairs
priority_queue<pair<int,int>, vector<pair<int,int>>,
greater<pair<int,int>>> pq;
```

Time Complexity:

- Insert: $O(\log n)$
- Delete: $O(\log n)$
- Top: $O(1)$

Use Cases: Dijkstra's algorithm, Kth largest/smallest, merge K sorted arrays

10 Pair & Tuple

Pair

What: Do elements ko ek sath store karta hai.

Example: `pair<int, string> p = {1, "hello"}` stores number and string together.

```

#include <utility>

// Declaration
pair<int, int> p;
pair<int, string> p = {1, "hello"};
pair<int, int> p = make_pair(10, 20);

// Access
cout << p.first;    // 1
cout << p.second;   // "hello"

// Update
p.first = 99;
p.second = "bye";

// Comparison (compares first, then second)
pair<int, int> p1 = {1, 2};
pair<int, int> p2 = {1, 3};
if(p1 < p2) cout << "p1 is smaller";

// In vectors
vector<pair<int, int>> v;
v.push_back({1, 2});
v.push_back(make_pair(3, 4));

// Sorting pairs
sort(v.begin(), v.end()); // sorts by first, then second

```

Tuple

What: Teen ya zyada elements ko ek sath store karta hai.

Example: `tuple<int, string, double> t = {1, "hello", 3.14}` stores 3 different types.

```
#include <tuple>

// Declaration
tuple<int, string, double> t;
tuple<int, string, double> t = make_tuple(1, "hello", 3.14);
tuple<int, int, int> t = {1, 2, 3}; // C++17

// Access
cout << get<0>(t); // 1
cout << get<1>(t); // "hello"
cout << get<2>(t); // 3.14

// Update
get<0>(t) = 99;

// Unpacking (C++17)
auto [a, b, c] = t;
cout << a << " " << b << " " << c;

// Size
int size = tuple_size<decltype(t)>::value; // 3
```

Time Complexity: All operations $O(1)$

Use Cases: Returning multiple values from function, storing 3+ related data

Quick Comparison Table

Data Structure	Ordered?	Duplicates?	Insert	Delete	Search	Access
vector	Yes	Yes	$O(1)$ end, $O(n)$ mid	$O(1)$ end, $O(n)$ mid	$O(n)$	$O(1)$
list	Yes	Yes	$O(1)$ ends	$O(1)$ ends	$O(n)$	$O(n)$
stack	-	Yes	$O(1)$	$O(1)$	$O(n)$	$O(1)$ top
queue	-	Yes	$O(1)$	$O(1)$	$O(n)$	$O(1)$ front/back
deque	Yes	Yes	$O(1)$ ends	$O(1)$ ends	$O(n)$	$O(1)$
unordered_map	No	No (keys)	$O(1)$ avg	$O(1)$ avg	$O(1)$ avg	$O(1)$ avg
map	Yes	No (keys)	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(\log n)$
unordered_set	No	No	$O(1)$ avg	$O(1)$ avg	$O(1)$ avg	-
set	Yes	No	$O(\log n)$	$O(\log n)$	$O(\log n)$	-
priority_queue	Partial	Yes	$O(\log n)$	$O(\log n)$	$O(n)$	$O(1)$ top

Interview Tips

1. **Hashing problems** → `unordered_map` or `unordered_set`
2. **Sorted data needed** → `set` or `map`
3. **Min/Max operations** → `priority_queue`
4. **LIFO operations** → `stack`
5. **FIFO operations** → `queue`
6. **Both ends access** → `deque`
7. **Dynamic array** → `vector`

- 8. **Frequency counting** → `unordered_map<int, int>`
 - 9. **Remove duplicates** → `set` or `unordered_set`
 - 10. **Top K elements** → `priority_queue`
-

🔥 **Pro Tip:** Most interview problems me `vector`, `unordered_map`, `unordered_set`, aur `priority_queue` ka 80% use hota hai. Inhe achhe se master kar lo! 100

07/01/2026, 13:24:49

👤 Yes but before that guide one by one proper and deep

07/01/2026, 13:25:25

🚀 C++ DSA Deep Dive Guide (One by One)

1 ARRAYS (Static Array)

📖 What is Array?

Array ek **fixed-size** container hai jo **same data type** ke multiple elements ko **contiguous memory** me store karta hai. Iska size compile time pe fix hota hai aur change nahi ho sakta.

Real Life Example: Ek parking lot jisme **exactly 10 slots** hai. Slot number 0 se 9 tak hai. Ek baar banane ke baad slots ki sankhya change nahi ho sakti.

🎯 How It Works Internally?

Memory Layout:

```
arr[5] = {10, 20, 30, 40, 50}
```

Index:	0	1	2	3	4
--------	---	---	---	---	---

Value:	10	20	30	40	50
--------	----	----	----	----	----

Address:	100	104	108	112	116	(each int = 4 bytes)
----------	-----	-----	-----	-----	-----	----------------------

Key Points:

- Elements **sequential memory** me store hote hai
- `arr[i]` ka matlab: **base_address + (i × size_of_element)**
- Isliye access **O(1)** time me hota hai
- Size fixed hai, badha nahi sakte



Complete Operations with Code

1 Declaration & Initialization

```
// Method 1: Size declare, values nahi
int arr[5]; // garbage values honge

// Method 2: Size + values
int arr[5] = {10, 20, 30, 40, 50};

// Method 3: Partial initialization (baki 0 honge)
int arr[5] = {10, 20}; // {10, 20, 0, 0, 0}

// Method 4: All zeros
int arr[5] = {0}; // {0, 0, 0, 0, 0}

// Method 5: Size auto-calculate
int arr[] = {10, 20, 30}; // size = 3

// Method 6: Character array
char arr[6] = "hello"; // {'h', 'e', 'l', 'l', 'o', '\0'}

// Method 7: 2D Array
int arr[3][4]; // 3 rows, 4 columns
int arr[2][3] = {{1, 2, 3}, {4, 5, 6}};
```



Example:

```
#include <iostream>
using namespace std;

int main() {
    int marks[5] = {85, 90, 78, 92, 88};

    cout << "First student marks: " << marks[0] << endl; // 85
    cout << "Array size: " << sizeof(marks)/sizeof(marks[0]) <<
endl; // 5

    return 0;
}
```

2 Access Elements - O(1)

```
int arr[5] = {10, 20, 30, 40, 50};

// Read
cout << arr[0]; // 10 (first element)
cout << arr[4]; // 50 (last element)
cout << arr[2]; // 30 (any index)

// Update
arr[2] = 99; // 30 ko 99 se replace karo
arr[0] = arr[0] + 5; // 10 ko 15 banao

// ⚠ Out of bounds (NO ERROR in C++, but dangerous)
cout << arr[10]; // undefined behavior
arr[-1] = 5; // dangerous, memory corruption
```

Example:

```
int scores[3] = {100, 200, 300};

// Access all
cout << "Score 1: " << scores[0] << endl; // 100
cout << "Score 2: " << scores[1] << endl; // 200
cout << "Score 3: " << scores[2] << endl; // 300

// Update
scores[1] = 250; // 200 -> 250
cout << "Updated Score 2: " << scores[1] << endl; // 250
```

3 Traversal (Iterate) - $O(n)$

```
int arr[5] = {10, 20, 30, 40, 50};
int n = 5;

// Method 1: Simple for loop
for(int i = 0; i < n; i++) {
    cout << arr[i] << " ";
}
// Output: 10 20 30 40 50

// Method 2: Range-based for loop (C++11)
for(int x : arr) {
    cout << x << " ";
}
// Output: 10 20 30 40 50

// Method 3: Reverse traversal
for(int i = n-1; i >= 0; i--) {
```

```

        cout << arr[i] << " ";
    }
    // Output: 50 40 30 20 10

    // Method 4: Pointer-based
    for(int* ptr = arr; ptr < arr + n; ptr++) {
        cout << *ptr << " ";
    }

```

Example: Sum of all elements

```

int arr[5] = {10, 20, 30, 40, 50};
int sum = 0;

for(int i = 0; i < 5; i++) {
    sum += arr[i];
}

cout << "Sum: " << sum; // 150

```

4 Insertion - O(n)

Problem: Array ka size fixed hai, toh technically insert nahi kar sakte. But agar space hai toh **shift** karke insert kar sakte ho.

```

#include <iostream>
using namespace std;

void insertAtPosition(int arr[], int &n, int pos, int value) {
    // Shift elements right from position
    for(int i = n; i > pos; i--) {
        arr[i] = arr[i-1];
    }
}

```

```

    }
    arr[pos] = value;
    n++; // size increase
}

int main() {
    int arr[10] = {10, 20, 30, 40, 50}; // max size 10
    int n = 5; // current elements

    // Insert 25 at index 2
    insertAtPosition(arr, n, 2, 25);

    // Print array
    for(int i = 0; i < n; i++) {
        cout << arr[i] << " ";
    }
    // Output: 10 20 25 30 40 50

    return 0;
}

```

Visual:

Before: {10, 20, 30, 40, 50}

Insert 25 at index 2

Step 1: Shift 50 -> index 5

Step 2: Shift 40 -> index 4

Step 3: Shift 30 -> index 3

Step 4: Insert 25 at index 2

After: {10, 20, 25, 30, 40, 50}

5 Deletion - O(n)

```
#include <iostream>
using namespace std;

void deleteAtPosition(int arr[], int &n, int pos) {
    // Shift elements left from position
    for(int i = pos; i < n-1; i++) {
        arr[i] = arr[i+1];
    }
    n--; // size decrease
}

int main() {
    int arr[10] = {10, 20, 30, 40, 50};
    int n = 5;

    // Delete element at index 2 (value 30)
    deleteAtPosition(arr, n, 2);

    // Print array
    for(int i = 0; i < n; i++) {
        cout << arr[i] << " ";
    }
    // Output: 10 20 40 50

    return 0;
}
```

Visual:

Before: {10, 20, 30, 40, 50}

Delete index 2 (30)

Step 1: Move 40 -> index 2

Step 2: Move 50 -> index 3

Step 3: Decrease size

After: {10, 20, 40, 50}

6 Search - $O(n)$ Linear, $O(\log n)$ Binary

Linear Search

```
int linearSearch(int arr[], int n, int key) {
    for(int i = 0; i < n; i++) {
        if(arr[i] == key) {
            return i; // found at index i
        }
    }
    return -1; // not found
}

// Usage
int arr[5] = {10, 20, 30, 40, 50};
int index = linearSearch(arr, 5, 30);
if(index != -1) {
    cout << "Found at index " << index; // Found at index 2
} else {
    cout << "Not found";
}
```

Using STL find() - $O(n)$

```
#include <algorithm>

int arr[5] = {10, 20, 30, 40, 50};

int* ptr = find(arr, arr + 5, 30);
```

```

if(ptr != arr + 5) {
    cout << "Found at index " << (ptr - arr); // 2
} else {
    cout << "Not found";
}

```

Binary Search (Array must be SORTED) - $O(\log n)$

```

#include <algorithm>
int arr[5] = {10, 20, 30, 40, 50}; // sorted

bool found = binary_search(arr, arr + 5, 30);
if(found) {
    cout << "Element exists";
}

// Get position
int* ptr = lower_bound(arr, arr + 5, 30);
cout << "Position: " << (ptr - arr); // 2

```

7 Sort - $O(n \log n)$

```

#include <algorithm>
int arr[5] = {50, 20, 40, 10, 30};

// Ascending order
sort(arr, arr + 5);
// Result: {10, 20, 30, 40, 50}

// Descending order
sort(arr, arr + 5, greater<int>());

```

```
// Result: {50, 40, 30, 20, 10}

// Sort only part of array
sort(arr + 1, arr + 4); // sort index 1 to 3
```

Example:

```
int marks[6] = {85, 92, 78, 95, 88, 90};

// Sort in ascending
sort(marks, marks + 6);
cout << "Sorted: ";
for(int x : marks) cout << x << " ";
// Output: 78 85 88 90 92 95

// Find highest and lowest
cout << "\nHighest: " << marks[5]; // 95
cout << "\nLowest: " << marks[0];  // 78
```

8 Reverse - O(n)

```
#include <algorithm>
int arr[5] = {10, 20, 30, 40, 50};

// Reverse entire array
reverse(arr, arr + 5);
// Result: {50, 40, 30, 20, 10}

// Reverse part of array
reverse(arr + 1, arr + 4); // reverse index 1 to 3
```


Manual Reverse:

```
void reverseArray(int arr[], int n) {
    int start = 0, end = n-1;
    while(start < end) {
        swap(arr[start], arr[end]);
        start++;
        end--;
    }
}

int arr[5] = {10, 20, 30, 40, 50};
reverseArray(arr, 5);
// Result: {50, 40, 30, 20, 10}
```

9 Size Calculation

```
int arr[5] = {10, 20, 30, 40, 50};

// Total bytes in array
int totalBytes = sizeof(arr); // 5 * 4 = 20 bytes

// Size of one element
int oneElement = sizeof(arr[0]); // 4 bytes

// Number of elements
int n = sizeof(arr) / sizeof(arr[0]); // 20/4 = 5

cout << "Array size: " << n; // 5
```

10 Common Utility Operations

```
#include <algorithm>

int arr[5] = {10, 20, 30, 40, 50};

// Fill array with value
fill(arr, arr + 5, 0); // {0, 0, 0, 0, 0}

// Copy array
int arr2[5];
copy(arr, arr + 5, arr2);

// Find min/max
int minVal = *min_element(arr, arr + 5);
int maxVal = *max_element(arr, arr + 5);

// Count occurrences
int cnt = count(arr, arr + 5, 30);

// Sum of elements
int sum = accumulate(arr, arr + 5, 0); // 0 is initial value
```

Time Complexity Summary

Operation	Time Complexity	Space
Access (arr[i])	O(1)	O(1)
Insert at end	O(1)	-
Insert at middle	O(n)	-
Delete from end	O(1)	-
Delete from middle	O(n)	-

Operation	Time Complexity	Space
Search (Linear)	$O(n)$	$O(1)$
Search (Binary)	$O(\log n)$	$O(1)$
Sort	$O(n \log n)$	$O(1)$
Reverse	$O(n)$	$O(1)$
Traversal	$O(n)$	$O(1)$

Interview Problems with Arrays

Problem 1: Find Maximum and Minimum

```
int arr[5] = {50, 20, 40, 10, 30};

int maxVal = arr[0], minVal = arr[0];

for(int i = 1; i < 5; i++) {
    if(arr[i] > maxVal) maxVal = arr[i];
    if(arr[i] < minVal) minVal = arr[i];
}

cout << "Max: " << maxVal << endl; // 50
cout << "Min: " << minVal << endl; // 10

// OR using STL
cout << "Max: " << *max_element(arr, arr+5) << endl;
cout << "Min: " << *min_element(arr, arr+5) << endl;
```

Problem 2: Reverse Array

```
int arr[5] = {10, 20, 30, 40, 50};
int n = 5;

int start = 0, end = n-1;
while(start < end) {
    swap(arr[start], arr[end]);
    start++;
    end--;
}

// Result: {50, 40, 30, 20, 10}
```

Problem 3: Check if Sorted

```
bool isSorted(int arr[], int n) {
    for(int i = 1; i < n; i++) {
        if(arr[i] < arr[i-1]) {
            return false;
        }
    }
    return true;
}

int arr1[5] = {10, 20, 30, 40, 50};
int arr2[5] = {10, 5, 30, 40, 50};

cout << isSorted(arr1, 5); // 1 (true)
cout << isSorted(arr2, 5); // 0 (false)
```

Problem 4: Left Rotate by 1

```
void leftRotate(int arr[], int n) {
    int first = arr[0];
    for(int i = 0; i < n-1; i++) {
        arr[i] = arr[i+1];
    }
    arr[n-1] = first;
}

int arr[5] = {10, 20, 30, 40, 50};
leftRotate(arr, 5);
// Result: {20, 30, 40, 50, 10}
```

Problem 5: Remove Duplicates (Sorted Array)

```
int removeDuplicates(int arr[], int n) {
    if(n == 0) return 0;

    int j = 0; // unique elements index
    for(int i = 1; i < n; i++) {
        if(arr[i] != arr[j]) {
            j++;
            arr[j] = arr[i];
        }
    }
    return j + 1; // new size
}

int arr[7] = {10, 10, 20, 20, 30, 30, 30};
int newSize = removeDuplicates(arr, 7);
```

```
for(int i = 0; i < newSize; i++) {  
    cout << arr[i] << " ";  
}  
  
// Output: 10 20 30
```

⚠ Common Mistakes & Tips

❌ Mistake 1: Array Index Out of Bounds

```
int arr[5] = {10, 20, 30, 40, 50};  
cout << arr[5]; // ❌ Index 5 doesn't exist (0-4 hai)  
cout << arr[10]; // ❌ Garbage value, crash possible
```

✅ Solution:

```
int n = sizeof(arr)/sizeof(arr[0]);  
if(index >= 0 && index < n) {  
    cout << arr[index]; // Safe  
}
```

❌ Mistake 2: Forgetting Array Size

```
void printArray(int arr[]) {  
    int n = sizeof(arr)/sizeof(arr[0]); // ❌ Wrong! Returns 1  
    for(int i = 0; i < n; i++) {  
        cout << arr[i];  
    }  
}
```

✅ Solution:

```
void printArray(int arr[], int n) { // Pass size as parameter
    for(int i = 0; i < n; i++) {
        cout << arr[i];
    }
}
```

❌ Mistake 3: Modifying Array While Iterating

```
int arr[5] = {10, 20, 30, 40, 50};
for(int i = 0; i < 5; i++) {
    arr[i+1] = arr[i] * 2; // ❌ Affects next iteration
}
```

🎓 When to Use Arrays?

✅ Use Arrays When:

- Size is **fixed** and known beforehand
- Need **fast random access** ($O(1)$)
- Memory me **continuous storage** chahiye
- Simple problems jaha size change nahi hota

❌ Don't Use Arrays When:

- Size **dynamic** hai (badalna/ghatna padega)
- Frequent **insertion/deletion** in middle

- Unknown size at compile time

For Dynamic Size → Use Vector (next topic!)

Shall I continue with **Vector** next? 🚀

07/01/2026, 13:26:32

2 VECTOR (Dynamic Array)



What is Vector?

Vector ek **dynamic array** hai jo **automatically resize** hota hai jab elements add/remove karte ho. Ye C++ STL (Standard Template Library) ka part hai.

Real Life Example: Ek **rubber band wala bag** - jitne chahiye utne items rakh sakte ho, bag automatically stretch ho jata hai. Static array ek **fixed box** hai jisme limited items hi rakh sakte ho.



How It Works Internally?

```
Initial capacity: 0
Add element 10 → capacity becomes 1
Add element 20 → capacity doubles to 2
Add element 30 → capacity doubles to 4
Add element 40 → capacity remains 4
Add element 50 → capacity doubles to 8
```

Memory Layout:

Index:	0	1	2	3	4
Value:	10	20	30	40	50

Key Concepts:

- **Size:** Current number of elements (5)



- **Capacity:** Total space allocated (8)
- Jab capacity full hoti hai, vector **naya memory allocate** karta hai (usually double) aur **purane elements copy** karta hai
- Isliye `push_back()` **O(1) amortized** hai (sometimes O(n) during resize)

Complete Operations with Code

1 Declaration & Initialization

```
#include <vector>
using namespace std;

// Method 1: Empty vector
vector<int> v; // size = 0, capacity = 0

// Method 2: With size (default value 0)
vector<int> v(5); // {0, 0, 0, 0, 0}

// Method 3: With size + value
vector<int> v(5, 10); // {10, 10, 10, 10, 10}

// Method 4: Initialize with values
vector<int> v = {10, 20, 30, 40, 50};

// Method 5: Copy from another vector
vector<int> v1 = {1, 2, 3};
vector<int> v2(v1); // v2 = {1, 2, 3}
vector<int> v3 = v1; // Same as above

// Method 6: From array
int arr[] = {10, 20, 30};
vector<int> v(arr, arr + 3);
```

```
// Method 7: 2D Vector
vector<vector<int>> v2d; // Empty 2D vector
vector<vector<int>> v2d(3, vector<int>(4, 0)); // 3x4 matrix
with 0s

// Method 8: Vector of pairs
vector<pair<int, int>> v = {{1,2}, {3,4}};

// Method 9: Vector of strings
vector<string> v = {"hello", "world"};
```

Example:

```
#include <iostream>
#include <vector>
using namespace std;

int main() {
    vector<int> marks = {85, 90, 78, 92, 88};

    cout << "Size: " << marks.size() << endl; // 5
    cout << "Capacity: " << marks.capacity() << endl; // 5 (or
more)
    cout << "First mark: " << marks[0] << endl; // 85

    return 0;
}
```

2 Insertion (Add Elements)

a) Insert at End - O(1) amortized

```
vector<int> v;

// push_back() - adds at end
v.push_back(10); // {10}
v.push_back(20); // {10, 20}
v.push_back(30); // {10, 20, 30}

// emplace_back() - faster (constructs in place)
v.emplace_back(40); // {10, 20, 30, 40}

cout << "Size: " << v.size(); // 4
```

Example:

```
vector<int> scores;

// Taking input
for(int i = 0; i < 5; i++) {
    int score;
    cin >> score;
    scores.push_back(score);
}

// Input: 100 85 92 78 95
// Vector: {100, 85, 92, 78, 95}
```

b) Insert at Specific Position - $O(n)$

```
vector<int> v = {10, 20, 30, 40};

// insert(position, value)
```

```
v.insert(v.begin(), 5);           // Insert 5 at beginning
// Result: {5, 10, 20, 30, 40}

v.insert(v.begin() + 2, 15);      // Insert 15 at index 2
// Result: {5, 10, 15, 20, 30, 40}

v.insert(v.end(), 50);            // Insert 50 at end (same as
push_back)
// Result: {5, 10, 15, 20, 30, 40, 50}

// Insert multiple elements
v.insert(v.begin() + 2, 3, 99);    // Insert 3 copies of 99 at
index 2
// Result: {5, 10, 99, 99, 99, 15, 20, 30, 40, 50}

// Insert from another vector
vector<int> v2 = {100, 200};
v.insert(v.end(), v2.begin(), v2.end()); // Append v2 to v
```

Visual Example:

Initial: {10, 20, 30, 40}

Insert 25 at index 2

Step 1: Shift elements right from index 2

{10, 20, _, 30, 40} (30, 40 shift right)

Step 2: Insert 25 at index 2

{10, 20, 25, 30, 40}

3 Deletion (Remove Elements)

a) Delete from End - $O(1)$

```
vector<int> v = {10, 20, 30, 40, 50};
```

```
v.pop_back(); // Removes 50  
// Result: {10, 20, 30, 40}
```

```
v.pop_back(); // Removes 40  
// Result: {10, 20, 30}
```

```
// Check if empty before pop_back  
if(!v.empty()) {  
    v.pop_back();  
}
```

b) Delete from Specific Position - $O(n)$

```
vector<int> v = {10, 20, 30, 40, 50};
```

```
// erase(position) - delete single element  
v.erase(v.begin() + 2); // Delete element at index 2 (30)  
// Result: {10, 20, 40, 50}
```

```
// erase(start, end) - delete range [start, end)  
v.erase(v.begin() + 1, v.begin() + 3); // Delete index 1 and 2  
// Result: {10, 50}
```

Example:

```
vector<int> v = {10, 20, 30, 40, 50};
```

```
// Delete first element
```

```
v.erase(v.begin());  
// Result: {20, 30, 40, 50}  
  
// Delete last element (better to use pop_back)  
v.erase(v.end() - 1);  
// Result: {20, 30, 40}  
  
// Delete first 2 elements  
v.erase(v.begin(), v.begin() + 2);  
// Result: {40}
```

c) Clear All Elements - $O(n)$

```
vector<int> v = {10, 20, 30, 40, 50};  
  
v.clear(); // Removes all elements  
// Result: {} (size = 0, but capacity remains)  
  
cout << "Size: " << v.size(); // 0  
cout << "Capacity: " << v.capacity(); // May still be > 0
```

4 Access Elements - $O(1)$

```
vector<int> v = {10, 20, 30, 40, 50};  
  
// Using [] operator (no bounds checking)  
cout << v[0]; // 10 (first element)  
cout << v[2]; // 30 (element at index 2)  
cout << v[4]; // 50 (last element)
```

```

// Using at() (with bounds checking, throws exception)
cout << v.at(0); // 10
cout << v.at(2); // 30

try {
    cout << v.at(10); // ✗ Throws out_of_range exception
} catch(exception &e) {
    cout << "Error: " << e.what();
}

// Access first and last
cout << v.front(); // 10 (first element)
cout << v.back(); // 50 (last element)

// Using pointers
int* ptr = v.data(); // Returns pointer to first element
cout << ptr[0]; // 10
cout << *(ptr + 2); // 30

```

Example:

```

vector<int> marks = {85, 90, 78, 92, 88};

// Print all marks
cout << "Mark 1: " << marks[0] << endl; // 85
cout << "Mark 2: " << marks[1] << endl; // 90
cout << "Mark 3: " << marks[2] << endl; // 78

// Print first and last
cout << "First: " << marks.front() << endl; // 85
cout << "Last: " << marks.back() << endl; // 88

```


5 Update Elements - O(1)

```
vector<int> v = {10, 20, 30, 40, 50};

// Update using []
v[0] = 15;    // {15, 20, 30, 40, 50}
v[2] = 35;    // {15, 20, 35, 40, 50}

// Update using at()
v.at(4) = 55; // {15, 20, 35, 40, 55}

// Update first and last
v.front() = 5;    // {5, 20, 35, 40, 55}
v.back() = 100;   // {5, 20, 35, 40, 100}

// Update using iterator
for(int i = 0; i < v.size(); i++) {
    v[i] = v[i] * 2; // Double all elements
}
// Result: {10, 40, 70, 80, 200}
```

6 Traversal (Iterate) - O(n)

```
vector<int> v = {10, 20, 30, 40, 50};

// Method 1: Using index
for(int i = 0; i < v.size(); i++) {
    cout << v[i] << " ";
}
// Output: 10 20 30 40 50

// Method 2: Range-based for loop (Best for reading)
```

```

for(int x : v) {
    cout << x << " ";
}
// Output: 10 20 30 40 50

// Method 3: Range-based with reference (for modification)
for(int &x : v) {
    x = x * 2; // Modifies original vector
}
// Result: {20, 40, 60, 80, 100}

// Method 4: Using iterators
for(auto it = v.begin(); it != v.end(); it++) {
    cout << *it << " ";
}
// Output: 20 40 60 80 100

// Method 5: Reverse iteration
for(int i = v.size() - 1; i >= 0; i--) {
    cout << v[i] << " ";
}
// Output: 100 80 60 40 20

// Method 6: Reverse iterator
for(auto it = v.rbegin(); it != v.rend(); it++) {
    cout << *it << " ";
}
// Output: 100 80 60 40 20

```

Example: Print even numbers only

```

vector<int> v = {10, 15, 20, 25, 30, 35};

for(int x : v) {
    if(x % 2 == 0) {

```

```
        cout << x << " ";
    }
}
// Output: 10 20 30
```

7 Search - $O(n)$ Linear, $O(\log n)$ Binary

Linear Search

```
vector<int> v = {10, 20, 30, 40, 50};

// Method 1: Manual search
int key = 30;
int index = -1;

for(int i = 0; i < v.size(); i++) {
    if(v[i] == key) {
        index = i;
        break;
    }
}

if(index != -1) {
    cout << "Found at index " << index; // Found at index 2
} else {
    cout << "Not found";
}

// Method 2: Using find() from <algorithm>
#include <algorithm>

auto it = find(v.begin(), v.end(), 30);
```

```

if(it != v.end()) {
    cout << "Found at index " << (it - v.begin()); // 2
} else {
    cout << "Not found";
}

// Check if element exists
if(find(v.begin(), v.end(), 30) != v.end()) {
    cout << "30 exists";
}

```

Binary Search (Vector MUST be sorted) - $O(\log n)$

```

#include <algorithm>
vector<int> v = {10, 20, 30, 40, 50}; // Sorted

// Method 1: binary_search() - returns true/false
bool found = binary_search(v.begin(), v.end(), 30);
if(found) {
    cout << "30 exists";
}

// Method 2: lower_bound() - first element >= key
auto it = lower_bound(v.begin(), v.end(), 30);
if(it != v.end()) {
    cout << "Found at index " << (it - v.begin()); // 2
}

// Method 3: upper_bound() - first element > key
auto it2 = upper_bound(v.begin(), v.end(), 30);
cout << "Index: " << (it2 - v.begin()); // 3 (points to 40)

```

Example: Find if element exists

```
vector<int> nums = {5, 12, 18, 23, 35, 47};

int target = 23;

if(binary_search(nums.begin(), nums.end(), target)) {
    cout << target << " found!";
} else {
    cout << target << " not found!";
}
```

8 Size & Capacity Operations

```
vector<int> v;

// Check if empty
if(v.empty()) {
    cout << "Vector is empty"; // True
}

// Add elements
v.push_back(10);
v.push_back(20);
v.push_back(30);

// Size - current number of elements
cout << "Size: " << v.size(); // 3

// Capacity - total allocated space
cout << "Capacity: " << v.capacity(); // >= 3 (usually 4)
```

```

// Resize vector
v.resize(5); // Size becomes 5, new elements = 0
// Result: {10, 20, 30, 0, 0}

v.resize(7, 100); // Size becomes 7, new elements = 100
// Result: {10, 20, 30, 0, 0, 100, 100}

v.resize(2); // Shrink to size 2
// Result: {10, 20}

// Reserve capacity (doesn't change size)
v.reserve(100); // Allocates space for 100 elements
cout << "Capacity: " << v.capacity(); // 100
cout << "Size: " << v.size(); // 2 (unchanged)

// Shrink capacity to fit size
v.shrink_to_fit();
cout << "Capacity: " << v.capacity(); // 2 (reduced)

// Max size (theoretical limit)
cout << v.max_size(); // Very large number

```

Example: Optimize memory

```

vector<int> v;

// Add 1000 elements
for(int i = 0; i < 1000; i++) {
    v.push_back(i);
}

cout << "Size: " << v.size(); // 1000
cout << "Capacity: " << v.capacity(); // > 1000 (maybe 1024)

// Now remove 900 elements

```

```
v.resize(100);

cout << "Size: " << v.size();           // 100
cout << "Capacity: " << v.capacity();    // Still 1024 (wasted
memory)

// Free unused memory
v.shrink_to_fit();
cout << "Capacity: " << v.capacity();    // 100 (optimized)
```

9 Sort & Reverse

Sort - $O(n \log n)$

```
#include <algorithm>
vector<int> v = {50, 20, 40, 10, 30};

// Sort ascending
sort(v.begin(), v.end());
// Result: {10, 20, 30, 40, 50}

// Sort descending
sort(v.begin(), v.end(), greater<int>());
// Result: {50, 40, 30, 20, 10}

// Sort part of vector
vector<int> v2 = {50, 20, 40, 10, 30};
sort(v2.begin() + 1, v2.begin() + 4); // Sort index 1 to 3
// Result: {50, 10, 20, 40, 30}
```

Custom Comparator:

```
// Sort pairs by second element
vector<pair<int, int>> v = {{1, 50}, {2, 20}, {3, 40}};

sort(v.begin(), v.end(), [](pair<int,int> a, pair<int,int> b) {
    return a.second < b.second;
});
// Result: {{2, 20}, {3, 40}, {1, 50}}

// Sort strings by length
vector<string> v2 = {"apple", "hi", "banana"};

sort(v2.begin(), v2.end(), [](string a, string b) {
    return a.size() < b.size();
});
// Result: {"hi", "apple", "banana"}
```

Reverse - O(n)

```
#include <algorithm>
vector<int> v = {10, 20, 30, 40, 50};

// Reverse entire vector
reverse(v.begin(), v.end());
// Result: {50, 40, 30, 20, 10}

// Reverse part of vector
vector<int> v2 = {10, 20, 30, 40, 50};
reverse(v2.begin() + 1, v2.begin() + 4); // Reverse index 1 to 3
// Result: {10, 40, 30, 20, 50}
```


10 Other Useful Operations

Copy Vector

```
vector<int> v1 = {10, 20, 30};

// Method 1: Using constructor
vector<int> v2(v1);

// Method 2: Using assignment
vector<int> v3 = v1;

// Method 3: Using assign()
vector<int> v4;
v4.assign(v1.begin(), v1.end());
```

Swap Vectors

```
vector<int> v1 = {10, 20, 30};
vector<int> v2 = {100, 200};

v1.swap(v2);

// v1 = {100, 200}
// v2 = {10, 20, 30}
```

Fill with Value

```
#include <algorithm>

vector<int> v(5);
```

```
fill(v.begin(), v.end(), 99);  
// Result: {99, 99, 99, 99, 99}
```

Count Element

```
#include <algorithm>  
  
vector<int> v = {10, 20, 10, 30, 10};  
  
int cnt = count(v.begin(), v.end(), 10);  
cout << "Count of 10: " << cnt; // 3
```

Sum of Elements

```
#include <numeric>  
  
vector<int> v = {10, 20, 30, 40, 50};  
  
int sum = accumulate(v.begin(), v.end(), 0); // 0 is initial  
value  
cout << "Sum: " << sum; // 150
```

Min/Max Element

```
#include <algorithm>  
  
vector<int> v = {10, 50, 20, 40, 30};
```

```

int minVal = *min_element(v.begin(), v.end());
int maxVal = *max_element(v.begin(), v.end());

cout << "Min: " << minVal;    // 10
cout << "Max: " << maxVal;    // 50

// Get index
auto minIt = min_element(v.begin(), v.end());
cout << "Min index: " << (minIt - v.begin()); // 0

```

Time Complexity Summary

Operation	Time Complexity	Explanation
push_back()	$O(1)$ amortized	Usually $O(1)$, $O(n)$ during resize
pop_back()	$O(1)$	Constant time
insert() at middle	$O(n)$	Shift elements needed
erase() at middle	$O(n)$	Shift elements needed
Access (v[i])	$O(1)$	Direct memory access
Search (linear)	$O(n)$	Check each element
Search (binary)	$O(\log n)$	Must be sorted
Sort	$O(n \log n)$	Comparison-based
Reverse	$O(n)$	Swap elements
clear()	$O(n)$	Destroy all elements
resize()	$O(n)$	May need reallocation

Interview Problems with Vectors

Problem 1: Remove Duplicates

```
#include <algorithm>

vector<int> v = {10, 20, 10, 30, 20, 40};

// Sort first
sort(v.begin(), v.end());
// {10, 10, 20, 20, 30, 40}

// Use unique() - moves duplicates to end, returns iterator
auto it = unique(v.begin(), v.end());

// Erase duplicates
v.erase(it, v.end());

// Result: {10, 20, 30, 40}
```

Problem 2: Find Second Largest

```
vector<int> v = {10, 50, 20, 40, 30};

// Sort descending
sort(v.begin(), v.end(), greater<int>());

cout << "Second largest: " << v[1]; // 40
```

Problem 3: Rotate Array

```
#include <algorithm>

vector<int> v = {10, 20, 30, 40, 50};

// Rotate left by 2 positions
rotate(v.begin(), v.begin() + 2, v.end());
// Result: {30, 40, 50, 10, 20}

// Rotate right by 2 positions
vector<int> v2 = {10, 20, 30, 40, 50};
rotate(v2.rbegin(), v2.rbegin() + 2, v2.rend());
// Result: {40, 50, 10, 20, 30}
```

Problem 4: Merge Two Sorted Vectors

```
vector<int> v1 = {10, 30, 50};
vector<int> v2 = {20, 40, 60};

vector<int> result;

// Use merge()
merge(v1.begin(), v1.end(), v2.begin(), v2.end(),
      back_inserter(result));

// Result: {10, 20, 30, 40, 50, 60}
```

Problem 5: Check if Vector is Palindrome



```

bool isPalindrome(vector<int> v) {
    int left = 0, right = v.size() - 1;

    while(left < right) {
        if(v[left] != v[right]) {
            return false;
        }
        left++;
        right--;
    }
    return true;
}

vector<int> v1 = {1, 2, 3, 2, 1};
vector<int> v2 = {1, 2, 3, 4, 5};

cout << isPalindrome(v1); // 1 (true)
cout << isPalindrome(v2); // 0 (false)

```

⚠ Common Mistakes & Tips

✗ Mistake 1: Modifying vector while iterating

```

vector<int> v = {10, 20, 30, 40, 50};

// ✗ Wrong - iterator becomes invalid
for(auto it = v.begin(); it != v.end(); it++) {
    if(*it == 30) {
        v.erase(it); // Iterator invalidated!
    }
}

```

✓ Solution:

```
// Correct way
for(auto it = v.begin(); it != v.end(); ) {
    if(*it == 30) {
        it = v.erase(it); // erase() returns next valid iterator
    } else {
        it++;
    }
}
```

✗ Mistake 2: Using v.size() in loop with unsigned

```
vector<int> v = {10, 20, 30};

// ✗ Infinite loop if i goes negative
for(int i = v.size() - 1; i >= 0; i--) { // size() returns
    cout << v[i];
}
```

✓ Solution:

```
// Use int casting
for(int i = (int)v.size() - 1; i >= 0; i--) {
    cout << v[i];
}
```

❌ Mistake 3: Not reserving capacity

```
vector<int> v;

// ❌ Multiple reallocations
for(int i = 0; i < 10000; i++) {
    v.push_back(i); // May reallocate many times
}
```

✅ Solution:

```
vector<int> v;
v.reserve(10000); // Pre-allocate space

for(int i = 0; i < 10000; i++) {
    v.push_back(i); // No reallocation needed
}
```

🎓 Vector vs Array - When to Use?

Feature	Array	Vector
Size	Fixed at compile time	Dynamic
Memory	Stack (usually)	Heap
Resize	❌ Not possible	✅ Automatic
Insert/Delete	❌ Manual shift	✅ Built-in methods
Bounds check	❌ No	✅ at() method
STL algorithms	✅ Works	✅ Works
Overhead	None	Small (size, capacity)

Feature	Array	Vector
Performance	Slightly faster	Slightly slower

Conclusion: Use **Vector** in 95% of cases. Use array only when size is fixed and performance is critical.

Pro Tips

1. **Reserve capacity** if you know the size beforehand:

```
vector<int> v;  
v.reserve(1000); // Avoid multiple reallocations
```

2. **Use `emplace_back()`** instead of `push_back()` for objects:

```
vector<pair<int,int>> v;  
v.emplace_back(1, 2); // Faster than v.push_back({1,2})
```

3. **Clear vector properly:**

```
v.clear(); // Removes elements  
v.shrink_to_fit(); // Frees memory
```

4. **Use range-based loops** for cleaner code:

```
for(int x : v) cout << x; // Instead of for(int i=0;  
i<v.size(); i++)
```

5. **Check empty before accessing:**

```
if(!v.empty()) {  
    cout << v.front();  
}
```

Ready for **Strings** next? 🚀

07/01/2026, 13:28:20

3 STRINGS

What is String?

String ek **sequence of characters** hai jo text store karta hai. C++ me string ek class hai (`std::string`) jo character array ko manage karta hai with automatic memory handling.

Real Life Example: Ek **sentence** ya **word** - jaise "Hello World", "12345", "abc@gmail.com". String me letters, numbers, symbols sab store kar sakte ho.

How It Works Internally?

```
string s = "hello";
```

Internal representation:

Index: 0 1 2 3 4

Char: 'h' 'e' 'l' 'l' 'o' '\0' (null terminator - automatically managed)

Memory: Dynamically allocated like vector

Size: 5

Capacity: ≥ 5 (can be more)

Key Points:

- String internally ek **dynamic character array** hai

- Automatic memory management (no manual `new/delete`)
- **Mutable** (change kar sakte ho)
- **NULL terminated** internally but you don't manage it
- Random access in **O(1)** time

Complete Operations with Code

1 Declaration & Initialization

```
#include <string>
using namespace std;

// Method 1: Empty string
string s; // ""

// Method 2: Initialize with string literal
string s = "hello";
string s("hello");

// Method 3: Initialize with n copies of character
string s(5, 'a'); // "aaaaa"

// Method 4: Copy from another string
string s1 = "world";
string s2 = s1; // "world"
string s3(s1); // "world"

// Method 5: From substring
string s1 = "hello world";
string s2(s1, 0, 5); // "hello" (start, length)

// Method 6: From character array
```

```

char arr[] = "example";
string s(arr); // "example"

// Method 7: From part of character array
char arr[] = "programming";
string s(arr, 7); // "program" (first 7 chars)

// Method 8: Using + operator
string s = "hello" + string(" world"); // "hello world"

// Method 9: Empty string check
string s;
if(s.empty()) {
    cout << "Empty string";
}

```

Example:

```

#include <iostream>
#include <string>
using namespace std;

int main() {
    string name = "Rahul";
    string greeting = "Hello, " + name + "!";

    cout << greeting << endl; // Hello, Rahul!
    cout << "Length: " << greeting.length() << endl; // 13

    return 0;
}

```

2 Input & Output

```
#include <iostream>
#include <string>
using namespace std;

// Method 1: cin (reads till space/newline)
string s;
cin >> s;
// Input: "hello world"
// s = "hello" (stops at space)

// Method 2: getline() (reads entire line including spaces)
string s;
getline(cin, s);
// Input: "hello world"
// s = "hello world"

// Method 3: getline with delimiter
string s;
getline(cin, s, ','); // Reads till comma
// Input: "hello,world"
// s = "hello"

// Output
string s = "hello";
cout << s << endl; // hello

// Character by character output
for(char c : s) {
    cout << c << " "; // h e l l o
}
```

Example: Taking full name

```

#include <iostream>
#include <string>
using namespace std;

int main() {
    string firstName, lastName, fullName;

    // Method 1: Two separate words
    cin >> firstName >> lastName;
    // Input: Rahul Kumar

    fullName = firstName + " " + lastName;
    cout << fullName << endl; // Rahul Kumar

    // Method 2: Full line
    cin.ignore(); // Clear buffer
    getline(cin, fullName);
    // Input: Rahul Kumar Singh
    cout << fullName << endl; // Rahul Kumar Singh

    return 0;
}

```

3 Access & Update - O(1)

```

string s = "hello";

// Access using [] (no bounds checking)
cout << s[0];    // 'h'
cout << s[4];    // 'o'

// Access using at() (with bounds checking)

```

```

cout << s.at(0); // 'h'

try {
    cout << s.at(10); // ✗ Throws out_of_range exception
} catch(exception &e) {
    cout << "Error: " << e.what();
}

// Access first and last character
cout << s.front(); // 'h'
cout << s.back(); // 'o'

// Update character
s[0] = 'H'; // "Hello"
s.at(1) = 'a'; // "Hallo"

s.front() = 'J'; // "Jallo"
s.back() = 'y'; // "Jally"

// Access using pointers
char* ptr = &s[0];
cout << ptr[0]; // 'J'
cout << *(ptr+1); // 'a'

```

Example: Change vowels to uppercase

```

string s = "hello world";

for(int i = 0; i < s.length(); i++) {
    if(s[i] == 'a' || s[i] == 'e' || s[i] == 'i' ||
        s[i] == 'o' || s[i] == 'u') {
        s[i] = toupper(s[i]);
    }
}

```



```
cout << s; // hEllo wOrld
```

4 Length & Size

```
string s = "hello";

// Length of string
cout << s.length(); // 5
cout << s.size();    // 5 (same as length)

// Check if empty
if(s.empty()) {
    cout << "Empty";
} else {
    cout << "Not empty";
}

// Capacity (allocated memory)
cout << s.capacity(); // >= 5

// Max possible size
cout << s.max_size(); // Very large number

// Clear string
s.clear();
cout << s.length(); // 0
cout << s.empty();   // 1 (true)

// Resize string
string s2 = "hello";
```

```
s2.resize(10);           // "hello      " (padded with null)
s2.resize(3);            // "hel" (truncated)
```

Example:

```
string password;
cout << "Enter password: ";
cin >> password;

if(password.length() < 8) {
    cout << "Password too short! Must be at least 8 characters.";
} else {
    cout << "Password accepted.";
}
```

5 Concatenation (Join Strings)

```
string s1 = "hello";
string s2 = "world";

// Method 1: Using + operator
string s3 = s1 + " " + s2; // "hello world"

// Method 2: Using += operator
string s4 = "hello";
s4 += " "; // "hello "
s4 += "world"; // "hello world"

// Method 3: Using append()
string s5 = "hello";
s5.append(" world"); // "hello world"
```

```

// Append n copies of character
s5.append(3, '!'); // "hello world!!!"

// Append substring
string s6 = "programming";
string s7 = "hello";
s7.append(s6, 0, 4); // "helloprog" (append first 4 chars)

// Method 4: Using push_back() for single character
string s8 = "hell";
s8.push_back('o'); // "hello"

// Method 5: Concatenate with numbers (need conversion)
string s9 = "User";
int id = 123;
s9 = s9 + to_string(id); // "User123"

```

Example: Build full address

```

string street = "MG Road";
string city = "Mumbai";
string state = "Maharashtra";
int pincode = 400001;

string address = street + ", " + city + ", " +
                 state + " - " + to_string(pincode);

cout << address;
// MG Road, Mumbai, Maharashtra - 400001

```

6 Substring - O(n)

```

string s = "hello world";

// substr(start_index)
string sub1 = s.substr(0);      // "hello world" (entire string)
string sub2 = s.substr(6);      // "world" (from index 6 to end)

// substr(start_index, length)
string sub3 = s.substr(0, 5);   // "hello" (start at 0, length 5)
string sub4 = s.substr(6, 5);   // "world" (start at 6, length 5)
string sub5 = s.substr(3, 2);   // "lo" (start at 3, length 2)

// Extract first word
string sentence = "hello world programming";
string firstWord = sentence.substr(0, sentence.find(' '));
cout << firstWord;  // "hello"

// Extract last word
int lastSpace = sentence.rfind(' ');
string lastWord = sentence.substr(lastSpace + 1);
cout << lastWord;  // "programming"

```

Example: Extract email parts

```

string email = "rahul@gmail.com";

// Extract username (before @)
int atPos = email.find('@');
string username = email.substr(0, atPos);
cout << "Username: " << username << endl;  // rahul

// Extract domain (after @)
string domain = email.substr(atPos + 1);
cout << "Domain: " << domain << endl;  // gmail.com

```

```
// Extract domain name only (before .)
int dotPos = domain.find('.');
string domainName = domain.substr(0, dotPos);
cout << "Domain name: " << domainName << endl; // gmail
```

7 Find & Search - O(n×m)

```
string s = "hello world programming";

// find() - returns index of first occurrence (or string::npos if
// not found)
int pos1 = s.find("world");
if(pos1 != string::npos) {
    cout << "Found at index: " << pos1; // 6
}

// find() with starting position
int pos2 = s.find('o'); // 4 (first 'o')
int pos3 = s.find('o', pos2+1); // 7 (second 'o' starting from
// index 5)

// rfind() - find from right (last occurrence)
int pos4 = s.rfind('o'); // 19 (last 'o')

// find_first_of() - find first occurrence of any character from
// set
string vowels = "aeiou";
int pos5 = s.find_first_of(vowels); // 1 ('e' in "hello")

// find_last_of() - find last occurrence
int pos6 = s.find_last_of(vowels); // 20 ('i' in "programming")
```

```

// find_first_not_of() - find first character NOT in set
int pos7 = s.find_first_not_of("helo "); // 6 ('w' in "world")

// Check if substring exists
if(s.find("world") != string::npos) {
    cout << "Substring found";
}

// Count occurrences
string text = "hello hello world";
string word = "hello";
int count = 0;
size_t pos = 0;

while((pos = text.find(word, pos)) != string::npos) {
    count++;
    pos += word.length();
}

cout << "Count: " << count; // 2

```

Example: Check if valid email

```

bool isValidEmail(string email) {
    // Must contain @ and .
    if(email.find('@') == string::npos) return false;
    if(email.find('.') == string::npos) return false;

    // @ should come before .
    int atPos = email.find('@');
    int dotPos = email.find('.', atPos);

    if(dotPos == string::npos) return false;

    return true;
}

```

```
cout << isValidEmail("test@gmail.com"); // 1 (true)
cout << isValidEmail("testgmail.com"); // 0 (false)
cout << isValidEmail("test@gmailcom"); // 0 (false)
```

8 Replace - O(n)

```
string s = "hello world";

// replace(start_pos, length, new_string)
s.replace(0, 5, "hi"); // "hi world" (replace "hello" with "hi")

s = "hello world";
s.replace(6, 5, "earth"); // "hello earth" (replace "world")

// Replace single character
s = "hello";
s.replace(1, 1, "a"); // "hallo"

// Replace all occurrences manually
string text = "I love cats. Cats are cute.";
string from = "cats";
string to = "dogs";

size_t pos = 0;
while((pos = text.find(from, pos)) != string::npos) {
    text.replace(pos, from.length(), to);
    pos += to.length();
}
cout << text; // "I love dogs. Cats are cute." (note: case-sensitive)
```

Example: Censor bad words

```
string censorWord(string text, string badWord) {
    string censored(badWord.length(), '*');
    size_t pos = 0;

    while((pos = text.find(badWord, pos)) != string::npos) {
        text.replace(pos, badWord.length(), censored);
        pos += censored.length();
    }

    return text;
}

string comment = "This is bad and very bad content";
comment = censorWord(comment, "bad");
cout << comment; // "This is *** and very *** content"
```

9 Insert & Erase

Insert - O(n)

```
string s = "hello world";

// insert(position, string)
s.insert(5, ","); // "hello, world" (insert at index 5)

s = "hello world";
s.insert(6, "beautiful "); // "hello beautiful world"

// insert(position, n_copies, character)
s = "hello";
s.insert(5, 3, '!'); // "hello!!!"
```



```
// Insert at beginning
s = "world";
s.insert(0, "hello "); // "hello world"

// Insert at end (better to use append or +=)
s.insert(s.length(), "!"); // "hello world!"
```

Erase - O(n)

```
string s = "hello world";

// erase(position, length)
s.erase(5, 6); // "hello" (remove " world")

s = "hello world";
s.erase(5, 1); // "helloworld" (remove space)

// erase(position) - removes from position to end
s = "hello world";
s.erase(5); // "hello"

// erase() - removes all
s = "hello";
s.erase(); // ""

// Remove last character
s = "hello!";
s.erase(s.length() - 1); // "hello"
// OR use pop_back()
s.pop_back(); // "hell"

// Remove first character
```

```
s = "hello";  
s.erase(0, 1); // "ello"  
// OR  
s.erase(s.begin()); // "llo"
```

Example: Remove all spaces

```
string s = "h e l l o   w o r l d";  
  
// Method 1: Using erase  
s.erase(remove(s.begin(), s.end(), ' '), s.end());  
cout << s; // "helloworld"  
  
// Method 2: Manual  
string s2 = "h e l l o   w o r l d";  
string result = "";  
for(char c : s2) {  
    if(c != ' ') {  
        result += c;  
    }  
}  
cout << result; // "helloworld"
```

10 Comparison

```
string s1 = "hello";  
string s2 = "world";  
string s3 = "hello";  
  
// Using == and != operators  
if(s1 == s3) {  
    cout << "Equal"; // ✓
```

```

}

if(s1 != s2) {
    cout << "Not equal"; // ✓
}

// Using compare() - returns 0 if equal, <0 if less, >0 if
// greater
int result = s1.compare(s2);
if(result == 0) {
    cout << "Equal";
} else if(result < 0) {
    cout << "s1 < s2"; // ✓ (lexicographically)
} else {
    cout << "s1 > s2";
}

// Lexicographic comparison using <, >, <=, >=
if(s1 < s2) {
    cout << "hello comes before world"; // ✓
}

// Compare substring
string s4 = "hello world";
if(s4.compare(0, 5, "hello") == 0) {
    cout << "Starts with hello"; // ✓
}

// Case-insensitive comparison (manual)
bool equalIgnoreCase(string s1, string s2) {
    if(s1.length() != s2.length()) return false;

    for(int i = 0; i < s1.length(); i++) {
        if(tolower(s1[i]) != tolower(s2[i])) {
            return false;
        }
    }
}

```

```

    }
}
return true;
}

cout << equalIgnoreCase("Hello", "hello"); // 1 (true)

```

11 Case Conversion

```

#include <algorithm>
#include <cctype>

string s = "Hello World";

// Convert to UPPERCASE
transform(s.begin(), s.end(), s.begin(), ::toupper);
cout << s; // "HELLO WORLD"

// Convert to lowercase
string s2 = "Hello World";
transform(s2.begin(), s2.end(), s2.begin(), ::tolower);
cout << s2; // "hello world"

// Manual conversion
string toUpper(string s) {
    for(int i = 0; i < s.length(); i++) {
        s[i] = toupper(s[i]);
    }
    return s;
}

string toLower(string s) {

```

```

    for(int i = 0; i < s.length(); i++) {
        s[i] = tolower(s[i]);
    }
    return s;
}

// Toggle case
string toggleCase(string s) {
    for(int i = 0; i < s.length(); i++) {
        if(isupper(s[i])) {
            s[i] = tolower(s[i]);
        } else {
            s[i] = toupper(s[i]);
        }
    }
    return s;
}

cout << toggleCase("Hello World"); // "hELLO wORLD"

```

1 2 Reverse String

```

#include <algorithm>

string s = "hello";

// Method 1: Using reverse()
reverse(s.begin(), s.end());
cout << s; // "olleh"

// Method 2: Manual (two pointers)
string reverseManual(string s) {

```

```

    int left = 0, right = s.length() - 1;

    while(left < right) {
        swap(s[left], s[right]);
        left++;
        right--;
    }

    return s;
}

// Method 3: Using loop
string reverseLoop(string s) {
    string result = "";
    for(int i = s.length() - 1; i >= 0; i--) {
        result += s[i];
    }
    return result;
}

// Reverse words in string
string reverseWords(string s) {
    string word = "", result = "";

    for(int i = 0; i < s.length(); i++) {
        if(s[i] == ' ') {
            result = word + " " + result;
            word = "";
        } else {
            word += s[i];
        }
    }

    result = word + " " + result;
    result.pop_back(); // Remove trailing space

```

```
        return result;
    }

    cout << reverseWords("hello world"); // "world hello"
```

13 Split String

```
#include <sstream>
#include <vector>

// Method 1: Using stringstream
vector<string> split(string s, char delimiter = ' ') {
    vector<string> tokens;
    stringstream ss(s);
    string token;

    while(getline(ss, token, delimiter)) {
        tokens.push_back(token);
    }

    return tokens;
}

string sentence = "hello,world,programming";
vector<string> words = split(sentence, ',');

for(string word : words) {
    cout << word << endl;
}

// Output:
// hello
// world
```

```

// programming

// Method 2: Manual split by space
vector<string> splitBySpace(string s) {
    vector<string> words;
    string word = "";

    for(char c : s) {
        if(c == ' ') {
            if(!word.empty()) {
                words.push_back(word);
                word = "";
            }
        } else {
            word += c;
        }
    }

    if(!word.empty()) {
        words.push_back(word);
    }

    return words;
}

// Method 3: Using strtok (C-style, not recommended for C++)
char str[] = "hello world programming";
char* token = strtok(str, " ");

while(token != NULL) {
    cout << token << endl;
    token = strtok(NULL, " ");
}

```


14 String to Number & Number to String

```
#include <string>

// String to Integer
string s1 = "123";
int num1 = stoi(s1); // 123

string s2 = "-456";
int num2 = stoi(s2); // -456

// String to Long
string s3 = "9876543210";
long num3 = stol(s3); // 9876543210

// String to Long Long
string s4 = "9876543210123";
long long num4 = stoll(s4); // 9876543210123

// String to Float
string s5 = "3.14";
float num5 = stof(s5); // 3.14

// String to Double
string s6 = "3.14159";
double num6 = stod(s6); // 3.14159

// Number to String
int n1 = 123;
string str1 = to_string(n1); // "123"

double d1 = 3.14;
string str2 = to_string(d1); // "3.140000"

// Handle exceptions
```

```
try {
    int num = stoi("abc"); // ❌ Not a number
} catch(invalid_argument &e) {
    cout << "Invalid input";
} catch(out_of_range &e) {
    cout << "Number too large";
}
```

Example: Sum of digits in string

```
string s = "a1b2c3";
int sum = 0;

for(char c : s) {
    if(isdigit(c)) {
        sum += (c - '0'); // Convert char to int
    }
}

cout << "Sum: " << sum; // 6 (1+2+3)
```

15 Character Checks

```
#include <cctype>

char c = 'A';

// Check if alphabetic
if(isalpha(c)) cout << "Letter"; // ✓

// Check if digit
if(isdigit(c)) cout << "Digit";
```

```

// Check if alphanumeric
if(isalnum(c)) cout << "Letter or digit"; // ✓

// Check if uppercase
if(isupper(c)) cout << "Uppercase"; // ✓

// Check if lowercase
if(islower(c)) cout << "Lowercase";

// Check if space
if(isspace(c)) cout << "Space";

// Check if punctuation
if ispunct('.') cout << "Punctuation"; // ✓

// Full example: Check password strength
bool isStrongPassword(string password) {
    if(password.length() < 8) return false;

    bool hasUpper = false, hasLower = false, hasDigit = false;

    for(char c : password) {
        if(isupper(c)) hasUpper = true;
        if(islower(c)) hasLower = true;
        if(isdigit(c)) hasDigit = true;
    }

    return hasUpper && hasLower && hasDigit;
}

cout << isStrongPassword("Pass123"); // 1 (true)
cout << isStrongPassword("password"); // 0 (false)

```

16 Traversal

```
string s = "hello";

// Method 1: Using index
for(int i = 0; i < s.length(); i++) {
    cout << s[i] << " ";
}
// Output: h e l l o

// Method 2: Range-based for loop
for(char c : s) {
    cout << c << " ";
}
// Output: h e l l o

// Method 3: Using iterators
for(auto it = s.begin(); it != s.end(); it++) {
    cout << *it << " ";
}
// Output: h e l l o

// Method 4: Reverse iteration
for(int i = s.length() - 1; i >= 0; i--) {
    cout << s[i] << " ";
}
// Output: o l l e h

// Method 5: Reverse iterator
for(auto it = s.rbegin(); it != s.rend(); it++) {
    cout << *it << " ";
}
// Output: o l l e h

// Modify while iterating
```

```
for(char &c : s) { // Note: reference (&) for modification
    c = toupper(c);
}
cout << s; // "HELLO"
```

17 Sort String

```
#include <algorithm>

string s = "hello";

// Sort ascending
sort(s.begin(), s.end());
cout << s; // "ehllo"

// Sort descending
string s2 = "hello";
sort(s2.begin(), s2.end(), greater<char>());
cout << s2; // "ollhe"

// Sort words in sentence
string sentence = "world hello programming";
vector<string> words = split(sentence); // Use split function
from above

sort(words.begin(), words.end());

for(string word : words) {
    cout << word << " ";
}
// Output: hello programming world
```

Time Complexity Summary

Operation	Time Complexity	Explanation
Access (s[i])	O(1)	Direct access
Append (+, push_back)	O(1) amortized	May need reallocation
Insert at middle	O(n)	Shift characters
Erase at middle	O(n)	Shift characters
Find/Search	O(n×m)	n = string length, m = pattern length
Substring	O(n)	Create new string
Replace	O(n)	Modify characters
Reverse	O(n)	Swap characters
Sort	O(n log n)	Comparison-based
Compare	O(n)	Character by character
Concatenation	O(n+m)	Create new string

Interview Problems with Strings

Problem 1: Check Palindrome

```
bool isPalindrome(string s) {  
    int left = 0, right = s.length() - 1;  
  
    while(left < right) {  
        if(s[left] != s[right]) {  
            return false;  
        }  
    }  
}
```

```

        left++;
        right--;
    }
    return true;
}

cout << isPalindrome("racecar"); // 1 (true)
cout << isPalindrome("hello");   // 0 (false)

```

Problem 2: Count Vowels and Consonants

```

void countVowelsConsonants(string s) {
    int vowels = 0, consonants = 0;

    for(char c : s) {
        c = tolower(c);
        if(c >= 'a' && c <= 'z') {
            if(c == 'a' || c == 'e' || c == 'i' || c == 'o' || c
== 'u') {
                vowels++;
            } else {
                consonants++;
            }
        }
    }

    cout << "Vowels: " << vowels << endl;
    cout << "Consonants: " << consonants << endl;
}

countVowelsConsonants("Hello World");

```

```
// Vowels: 3
// Consonants: 7
```

Problem 3: Remove Duplicates (Keep first occurrence)

```
string removeDuplicates(string s) {
    string result = "";
    bool seen[256] = {false}; // ASCII characters

    for(char c : s) {
        if(!seen[c]) {
            result += c;
            seen[c] = true;
        }
    }

    return result;
}

cout << removeDuplicates("hello"); // "helo"
cout << removeDuplicates("programming"); // "progamin"
```

Problem 4: Check Anagram

```
bool isAnagram(string s1, string s2) {
    if(s1.length() != s2.length()) return false;

    sort(s1.begin(), s1.end());
    sort(s2.begin(), s2.end());
```



```

        return s1 == s2;
    }

    cout << isAnagram("listen", "silent"); // 1 (true)
    cout << isAnagram("hello", "world");   // 0 (false)

    // Optimized: Using frequency count
    bool isAnagramOptimized(string s1, string s2) {
        if(s1.length() != s2.length()) return false;

        int count[256] = {0};

        for(int i = 0; i < s1.length(); i++) {
            count[s1[i]]++;
            count[s2[i]]--;
        }

        for(int i = 0; i < 256; i++) {
            if(count[i] != 0) return false;
        }
    }

```

07/01/2026, 14:10:13



3 STRINGS (Continued)

Problem 5: First Non-Repeating Character

```
char firstNonRepeating(string s) {  
    int freq[256] = {0}; // ASCII characters  
  
    // Count frequency  
    for(char c : s) {  
        freq[c]++;  
    }  
  
    // Find first with frequency 1  
    for(char c : s) {  
        if(freq[c] == 1) {  
            return c;  
        }  
    }  
  
    return '\0'; // No non-repeating character  
}  
  
cout << firstNonRepeating("hello"); // 'h'  
cout << firstNonRepeating("programming"); // 'p'  
cout << firstNonRepeating("aabbcc"); // '\0' (none)
```

Problem 6: Longest Common Prefix

```
string longestCommonPrefix(vector<string> strs) {  
    if(strs.empty()) return "";  
  
    string prefix = strs[0];  
  
    for(int i = 1; i < strs.size(); i++) {  
        while(strs[i].find(prefix) != 0) { // Not found at  
position 0  
            prefix = prefix.substr(0, prefix.length() - 1);  
            if(prefix.empty()) return "";  
        }  
    }  
  
    return prefix;  
}  
  
vector<string> words = {"flower", "flow", "flight"};  
cout << longestCommonPrefix(words); // "fl"  
  
vector<string> words2 = {"dog", "racecar", "car"};  
cout << longestCommonPrefix(words2); // ""
```

Problem 7: Valid Parentheses

```
bool isValidParentheses(string s) {  
    stack<char> st;  
  
    for(char c : s) {  
        if(c == '(' || c == '{' || c == '[') {  
            st.push(c);  
        }
```

```

    } else {
        if(st.empty()) return false;

        char top = st.top();
        if((c == ')' && top == '(') ||
           (c == '}' && top == '{') ||
           (c == ']' && top == '[')) {
            st.pop();
        } else {
            return false;
        }
    }

    return st.empty();
}

cout << isValidParentheses("()[{}]); // 1 (true)
cout << isValidParentheses("]"); // 0 (false)
cout << isValidParentheses("([]]"); // 0 (false)
cout << isValidParentheses("{[]}"); // 1 (true)

```

Problem 8: Compress String

```

string compressString(string s) {
    string result = "";
    int count = 1;

    for(int i = 0; i < s.length(); i++) {
        if(i + 1 < s.length() && s[i] == s[i + 1]) {
            count++;
        } else {

```

```

        result += s[i];
        if(count > 1) {
            result += to_string(count);
        }
        count = 1;
    }
}

// Return original if compressed is not smaller
return result.length() < s.length() ? result : s;
}

cout << compressString("aabbccccc");    // "a2b3c4"
cout << compressString("abc");           // "abc" (not compressed)
cout << compressString("aaabbbccc");     // "a3b3c3"

```

Problem 9: String Rotation Check

```

bool isRotation(string s1, string s2) {
    if(s1.length() != s2.length()) return false;

    // Concatenate s1 with itself
    string temp = s1 + s1;

    // Check if s2 is substring of temp
    return temp.find(s2) != string::npos;
}

cout << isRotation("waterbottle", "erbottlewat"); // 1 (true)
cout << isRotation("hello", "lohel");             // 1 (true)
cout << isRotation("hello", "world");             // 0 (false)

```

```
/* How it works:
   s1 = "waterbottle"
   s2 = "erbottlewat"

   temp = s1 + s1 = "waterbottlewaterbottle"

   s2 is substring of temp? YES!
   "waterbottle[erbottlewat]erbottle"
*/
```

Problem 10: Longest Palindromic Substring

```
string longestPalindrome(string s) {
    if(s.empty()) return "";

    int maxLen = 1;
    int start = 0;

    // Check for each center
    for(int i = 0; i < s.length(); i++) {
        // Odd length palindrome
        int left = i, right = i;
        while(left >= 0 && right < s.length() && s[left] ==
s[right]) {
            int len = right - left + 1;
            if(len > maxLen) {
                maxLen = len;
                start = left;
            }
            left--;
            right++;
        }
    }
}
```

```

        // Even length palindrome
        left = i;
        right = i + 1;
        while(left >= 0 && right < s.length() && s[left] ==
s[right]) {
            int len = right - left + 1;
            if(len > maxLen) {
                maxLen = len;
                start = left;
            }
            left--;
            right++;
        }
    }

    return s.substr(start, maxLen);
}

cout << longestPalindrome("babad");    // "bab" or "aba"
cout << longestPalindrome("cbbd");    // "bb"
cout << longestPalindrome("racecar"); // "racecar"

```

Problem 11: Remove Adjacent Duplicates

```

string removeAdjacentDuplicates(string s) {
    stack<char> st;

    for(char c : s) {
        if(!st.empty() && st.top() == c) {
            st.pop(); // Remove duplicate
        } else {

```

```

        st.push(c);
    }
}

// Build result from stack
string result = "";
while(!st.empty()) {
    result = st.top() + result;
    st.pop();
}

return result;
}

cout << removeAdjacentDuplicates("abbaca");    // "ca"
cout << removeAdjacentDuplicates("azxxzy");    // "ay"
cout << removeAdjacentDuplicates("aabbbaac");  // "c"

/* How it works:
    "abbaca"
    a -> stack: [a]
    b -> stack: [a, b]
    b -> duplicate! pop -> stack: [a]
    a -> duplicate! pop -> stack: []
    c -> stack: [c]
    a -> stack: [c, a]
    Result: "ca"
*/

```

Problem 12: Word Pattern Match


```

bool wordPattern(string pattern, string s) {
    vector<string> words;
    stringstream ss(s);
    string word;

    while(ss >> word) {
        words.push_back(word);
    }

    if(pattern.length() != words.size()) return false;

    unordered_map<char, string> charToWord;
    unordered_map<string, char> wordToChar;

    for(int i = 0; i < pattern.length(); i++) {
        char c = pattern[i];
        string w = words[i];

        if(charToWord.count(c) && charToWord[c] != w) return
false;
        if(wordToChar.count(w) && wordToChar[w] != c) return
false;

        charToWord[c] = w;
        wordToChar[w] = c;
    }

    return true;
}

cout << wordPattern("abba", "dog cat cat dog"); // 1 (true)
cout << wordPattern("abba", "dog cat cat fish"); // 0 (false)
cout << wordPattern("aaaa", "dog cat cat dog"); // 0 (false)

```

Problem 13: Zigzag Conversion

```
string zigzagConversion(string s, int numRows) {
    if(numRows == 1) return s;

    vector<string> rows(min(numRows, (int)s.length()));
    int currentRow = 0;
    bool goingDown = false;

    for(char c : s) {
        rows[currentRow] += c;

        if(currentRow == 0 || currentRow == numRows - 1) {
            goingDown = !goingDown;
        }

        currentRow += goingDown ? 1 : -1;
    }

    string result = "";
    for(string row : rows) {
        result += row;
    }

    return result;
}

cout << zigzagConversion("PAYPALISHIRING", 3);
// "PAHNAPLSIIGYIR"

/*
P   A   H   N
A P L S I I G
*/
```

Y I R

Read row by row: PAHNAPLSIIGYIR

*/

Problem 14: Integer to Roman

```
string intToRoman(int num) {
    vector<pair<int, string>> values = {
        {1000, "M"}, {900, "CM"}, {500, "D"}, {400, "CD"},
        {100, "C"}, {90, "XC"}, {50, "L"}, {40, "XL"},
        {10, "X"}, {9, "IX"}, {5, "V"}, {4, "IV"}, {1, "I"}
    };

    string result = "";

    for(auto [value, symbol] : values) {
        while(num >= value) {
            result += symbol;
            num -= value;
        }
    }

    return result;
}

cout << intToRoman(3);      // "III"
cout << intToRoman(58);     // "LVIII"
cout << intToRoman(1994);   // "MCMXCIV"
```

Problem 15: Roman to Integer

```
int romanToInt(string s) {
    unordered_map<char, int> romanValues = {
        {'I', 1}, {'V', 5}, {'X', 10}, {'L', 50},
        {'C', 100}, {'D', 500}, {'M', 1000}
    };

    int result = 0;

    for(int i = 0; i < s.length(); i++) {
        int current = romanValues[s[i]];

        if(i + 1 < s.length()) {
            int next = romanValues[s[i + 1]];

            if(current < next) {
                result -= current; // Subtraction case (IV, IX,
// XL, etc.)
            } else {
                result += current;
            }
        } else {
            result += current;
        }
    }

    return result;
}

cout << romanToInt("III"); // 3
cout << romanToInt("LVIII"); // 58
cout << romanToInt("MCMXCIV"); // 1994
```

⚠ Common Mistakes & Tips

❌ Mistake 1: Not checking empty string

```
string s = "";  
char c = s[0]; // ❌ Undefined behavior
```

✅ Solution:

```
if(!s.empty()) {  
    char c = s[0]; // Safe  
}
```

❌ Mistake 2: Modifying string while iterating

```
string s = "hello";  
for(int i = 0; i < s.length(); i++) {  
    if(s[i] == 'l') {  
        s.erase(i, 1); // ❌ Skips next character  
    }  
}
```

✅ Solution:

```
string s = "hello";  
for(int i = 0; i < s.length(); ) {  
    if(s[i] == 'l') {  
        s.erase(i, 1); // Don't increment i  
    } else {  
        i++;  
    }  
}
```

```
}  
}
```

✗ Mistake 3: Using == for case-insensitive comparison

```
string s1 = "Hello";  
string s2 = "hello";  
if(s1 == s2) { // ✗ False (case-sensitive)  
    cout << "Equal";  
}
```

✓ Solution:

```
// Convert both to lowercase first  
transform(s1.begin(), s1.end(), s1.begin(), ::tolower);  
transform(s2.begin(), s2.end(), s2.begin(), ::tolower);  
if(s1 == s2) { // ✓ True  
    cout << "Equal";  
}
```

✗ Mistake 4: Unnecessary string concatenation in loops

```
string result = "";  
for(int i = 0; i < 10000; i++) {  
    result += "a"; // ✗ Multiple reallocations  
}
```

✓ Solution:

```
string result;  
result.reserve(10000); // Pre-allocate  
for(int i = 0; i < 10000; i++) {  
    result += "a"; // No reallocation  
}  
  
// OR  
string result(10000, 'a'); // Direct construction
```

✗ Mistake 5: Forgetting null check in find()

```
string s = "hello";  
int pos = s.find("world");  
cout << s.substr(pos); // ✗ undefined behavior if not found
```

✓ Solution:

```
string s = "hello";  
int pos = s.find("world");  
if(pos != string::npos) {  
    cout << s.substr(pos);  
} else {  
    cout << "Not found";  
}
```

Pro Tips

1 Use string_view (C++17) for performance

```
#include <string_view>

void processString(string_view sv) {
    // No copying, just views the original string
    cout << sv.substr(0, 5); // Fast, no allocation
}

string s = "hello world";
processString(s); // No copy made
```

2 Use emplace instead of insert for objects

```
string s = "hello";
s.insert(5, 1, '!'); // Slower

// Better for complex operations
s.reserve(s.length() + 1);
s.push_back('!');
```

3 Clear string properly

```
string s = "hello world";

// Just clears content, capacity remains
s.clear();
cout << s.capacity(); // Still > 0
```



```
// Free memory too
s.clear();
s.shrink_to_fit();
cout << s.capacity(); // 0
```

4 Use const reference in loops to avoid copies

```
vector<string> words = {"hello", "world", "programming"};
```

```
// ❌ Copies each string
for(string word : words) {
    cout << word;
}
```

```
// ✅ No copy, read-only
for(const string& word : words) {
    cout << word;
}
```

```
// ✅ No copy, can modify original
for(string& word : words) {
    word[0] = toupper(word[0]);
}
```

5 Optimize substr usage

```
string s = "hello world";

// ❌ Creates new string (allocation)
```

```
string sub = s.substr(0, 5);

// ✅ Just view, no allocation (C++17)
string_view sv(s.data(), 5);
```

6 Use appropriate find method

```
string s = "hello world";

// Find single character
int pos1 = s.find('o');           // Fast

// Find string
int pos2 = s.find("world");       // Slower

// Find any of multiple characters
int pos3 = s.find_first_of("aeiou"); // Find first vowel
```

7 Efficient string building

```
// ❌ Slow for many concatenations
string result = "";
for(int i = 0; i < 1000; i++) {
    result += to_string(i) + " ";
}

// ✅ Fast
stringstream ss;
for(int i = 0; i < 1000; i++) {
```

```
    ss << i << " ";
}
string result = ss.str();

// OR pre-reserve
string result;
result.reserve(5000); // Estimate size
for(int i = 0; i < 1000; i++) {
    result += to_string(i) + " ";
}
```

8 Compare strings efficiently

```
string s1 = "hello";
string s2 = "world";

// ✅ Fast - compares pointers first
if(s1 == s2) { }

// ❌ Slower - always compares character by character
if(s1.compare(s2) == 0) { }

// Use compare() when you need ordering information
if(s1.compare(s2) < 0) {
    cout << "s1 comes before s2";
}
```

9 Handle special characters

```
string s = "hello\nworld\ttab";

// Check for newline
if(s.find('\n') != string::npos) {
    cout << "Contains newline";
}

// Remove all whitespace characters
s.erase(remove_if(s.begin(), s.end(), ::isspace), s.end());
```

10 Tokenization best practices

```
// For simple splitting
stringstream ss("hello world");
string word;
while(ss >> word) {
    cout << word << endl;
}

// For CSV or custom delimiters
string data = "name,age,city";
stringstream ss(data);
string token;

while(getline(ss, token, ',')) {
    cout << token << endl;
}
```

🎓 String vs char[] vs char*

Feature	string	char[]	char*
Type	Class	Array	Pointer
Size	Dynamic	Fixed	Fixed/Dynamic
Memory	Heap (managed)	Stack	Stack/Heap
Null terminator	Auto-managed	Manual	Manual
Functions	Many built-in	Limited	Limited
Safety	High	Medium	Low
Performance	Slightly slower	Fast	Fast
Recommended	✅ Yes	For small fixed	❌ Avoid

Conclusion: Use `string` in 99% of cases for safety and convenience.

📝 Summary Cheatsheet

```
// Creation
string s = "hello";
string s(5, 'a'); // "aaaaa"

// Size
s.length(), s.size() // 5
s.empty() // false

// Access
s[0], s.at(0) // 'h'
s.front(), s.back() // 'h', 'o'
```

```

// Modify
s += "world";           // Append
s.append("!");          // Append
s.insert(5, " ");       // Insert
s.erase(0, 5);          // Erase
s.replace(0, 5, "hi");   // Replace
s.clear();               // Clear all

// Search
s.find("world");         // Find substring
s.rfind('o');            // Find from end
s.find_first_of("aeiou"); // Find any vowel

// Substring
s.substr(0, 5);           // "hello"

// Compare
s1 == s2, s1 < s2         // Comparison
s1.compare(s2)            // -1, 0, 1

// Case
transform(s.begin(), s.end(), s.begin(), ::toupper);
transform(s.begin(), s.end(), s.begin(), ::tolower);

// Reverse & Sort
reverse(s.begin(), s.end());
sort(s.begin(), s.end());

// Conversion
stoi("123"), stol(), stod() // String to number
to_string(123)              // Number to string

// Character checks
isalpha(c), isdigit(c), isspace(c)

```

```
isupper(c), islower(c)  
toupper(c), tolower(c)
```

Ready for **Linked Lists** next? 🚀

07/01/2026, 14:11:27

4 LINKED LIST

What is Linked List?

Linked List ek linear data structure hai jisme elements (nodes) **non-contiguous memory** me store hote hai. Har node me **data** aur **pointer to next node** hota hai.

Real Life Example: Ek **train** - har coach (node) me passengers (data) hote hai aur next coach ka connection (pointer) hota hai. Train ka first coach HEAD hai, last coach ka pointer NULL hota hai.

Array vs Linked List:

- **Array:** Continuous memory, fixed size (parking lot with fixed slots)
- **Linked List:** Scattered memory, dynamic size (train me coaches add/remove kar sakte ho)

Types of Linked Lists

1 Singly Linked List

```
[Data|Next] -> [Data|Next] -> [Data|Next] -> NULL
```

Ek hi direction me traverse kar sakte ho (forward only)

2 Doubly Linked List


```
NULL <- [Prev|Data|Next] <-> [Prev|Data|Next] <->
[Prev|Data|Next] -> NULL
```

Dono direction me traverse kar sakte ho (forward & backward)

3 Circular Linked List

```
[Data|Next] -> [Data|Next] -> [Data|Next] -> (points back to
first)
```

Last node first node ko point karta hai



Implementation Options

Option 1: Manual Implementation (Custom Node)

Full control, but more code

Option 2: STL list<> (Doubly Linked List)

Built-in, easy to use, recommended for interviews

We'll cover **both approaches** 🚀



APPROACH 1: Manual Singly Linked List

Node Structure



```
struct Node {
    int data;
    Node* next;

    // Constructor
    Node(int val) {
        data = val;
        next = NULL;
    }
};
```

Visual:

```
Node* head = new Node(10);
```

Memory:

```
head -> [10 | NULL]
```

1 Creation & Initialization

```
#include <iostream>
using namespace std;

struct Node {
    int data;
    Node* next;

    Node(int val) : data(val), next(NULL) {}
};

int main() {
```

```

// Create single node
Node* head = new Node(10);

// Create multiple nodes
Node* second = new Node(20);
Node* third = new Node(30);

// Link nodes
head->next = second;
second->next = third;

// Now: head -> 10 -> 20 -> 30 -> NULL

return 0;
}

```

Visual:

```
head -> [10|•] -> [20|•] -> [30|NULL]
```

2 Insertion Operations

a) Insert at Beginning - $O(1)$

```

Node* insertAtBeginning(Node* head, int data) {
    Node* newNode = new Node(data);
    newNode->next = head;
    return newNode; // New head
}

// Usage
Node* head = NULL;

```

```
head = insertAtBeginning(head, 10); // 10 -> NULL
head = insertAtBeginning(head, 20); // 20 -> 10 -> NULL
head = insertAtBeginning(head, 30); // 30 -> 20 -> 10 -> NULL
```

Visual:

Before: head -> [20|•] -> [10|NULL]

Insert 30 at beginning:

Step 1: Create new node: [30|NULL]

Step 2: Point to current head: [30|•] -> [20|•] -> [10|NULL]

Step 3: Update head: head -> [30|•]

After: head -> [30|•] -> [20|•] -> [10|NULL]

b) Insert at End - O(n)

```
Node* insertAtEnd(Node* head, int data) {
    Node* newNode = new Node(data);

    // If list is empty
    if(head == NULL) {
        return newNode;
    }

    // Traverse to last node
    Node* temp = head;
    while(temp->next != NULL) {
        temp = temp->next;
    }

    // Link last node to new node
```

```

    temp->next = newNode;
    return head;
}

// Usage
Node* head = NULL;
head = insertAtEnd(head, 10); // 10 -> NULL
head = insertAtEnd(head, 20); // 10 -> 20 -> NULL
head = insertAtEnd(head, 30); // 10 -> 20 -> 30 -> NULL

```

Visual:

Before: head -> [10|•] -> [20|NULL]

Insert 30 at end:

Step 1: Traverse to last node (20)

Step 2: Create new node: [30|NULL]

Step 3: Link: [20|•] -> [30|NULL]

After: head -> [10|•] -> [20|•] -> [30|NULL]

c) Insert at Position - $O(n)$

```

Node* insertAtPosition(Node* head, int data, int pos) {
    Node* newNode = new Node(data);

    // Insert at beginning (position 0)
    if(pos == 0) {
        newNode->next = head;
        return newNode;
    }
}

```

```

// Traverse to position-1
Node* temp = head;
for(int i = 0; i < pos - 1 && temp != NULL; i++) {
    temp = temp->next;
}

// If position is invalid
if(temp == NULL) {
    cout << "Invalid position\n";
    delete newNode;
    return head;
}

// Insert node
newNode->next = temp->next;
temp->next = newNode;

return head;
}

// Usage
Node* head = new Node(10);
head->next = new Node(30);

// Insert 20 at position 1 (between 10 and 30)
head = insertAtPosition(head, 20, 1);
// Result: 10 -> 20 -> 30 -> NULL

```

Visual:

Before: head -> [10|•] -> [30|NULL]
 Position: 0 1

Insert 20 at position 1:

Step 1: Traverse to position 0 (node with 10)

Step 2: Create new node: [20|NULL]

Step 3: Point new node to next: [20|•] -> [30|NULL]

Step 4: Point previous to new: [10|•] -> [20|•]

After: head -> [10|•] -> [20|•] -> [30|NULL]

Position: 0 1 2

3 Deletion Operations

a) Delete from Beginning - O(1)

```
Node* deleteFromBeginning(Node* head) {
    if(head == NULL) {
        cout << "List is empty\n";
        return NULL;
    }

    Node* temp = head;
    head = head->next;
    delete temp;

    return head;
}
```

// Usage

```
Node* head = new Node(10);
head->next = new Node(20);
head->next->next = new Node(30);
// List: 10 -> 20 -> 30 -> NULL
```

```
head = deleteFromBeginning(head);
// List: 20 -> 30 -> NULL
```

Visual:

Before: head -> [10|•] -> [20|•] -> [30|NULL]

Delete from beginning:

Step 1: temp = head (pointing to 10)

Step 2: head = head->next (now pointing to 20)

Step 3: delete temp (free memory of 10)

After: head -> [20|•] -> [30|NULL]

b) Delete from End - O(n)

```
Node* deleteFromEnd(Node* head) {
    if(head == NULL) {
        cout << "List is empty\n";
        return NULL;
    }

    // If only one node
    if(head->next == NULL) {
        delete head;
        return NULL;
    }

    // Traverse to second last node
    Node* temp = head;
    while(temp->next->next != NULL) {
        temp = temp->next;
    }

    // Delete last node
```



```

    delete temp->next;
    temp->next = NULL;

    return head;
}

// Usage
Node* head = new Node(10);
head->next = new Node(20);
head->next->next = new Node(30);
// List: 10 -> 20 -> 30 -> NULL

head = deleteFromEnd(head);
// List: 10 -> 20 -> NULL

```

Visual:

Before: head -> [10|•] -> [20|•] -> [30|NULL]

Delete from end:

Step 1: Traverse to second last (20)

Step 2: Delete 30

Step 3: Set 20's next to NULL

After: head -> [10|•] -> [20|NULL]

c) Delete at Position - O(n)

```

Node* deleteAtPosition(Node* head, int pos) {
    if(head == NULL) {
        cout << "List is empty\n";
        return NULL;
    }
}

```

```

}

// Delete first node
if(pos == 0) {
    Node* temp = head;
    head = head->next;
    delete temp;
    return head;
}

// Traverse to position-1
Node* temp = head;
for(int i = 0; i < pos - 1 && temp != NULL; i++) {
    temp = temp->next;
}

// If position is invalid
if(temp == NULL || temp->next == NULL) {
    cout << "Invalid position\n";
    return head;
}

// Delete node
Node* nodeToDelete = temp->next;
temp->next = temp->next->next;
delete nodeToDelete;

return head;
}

// Usage
Node* head = new Node(10);
head->next = new Node(20);
head->next->next = new Node(30);
// List: 10 -> 20 -> 30 -> NULL

```

```
head = deleteAtPosition(head, 1); // Delete 20
// List: 10 -> 30 -> NULL
```

d) Delete by Value - O(n)

```
Node* deleteByValue(Node* head, int value) {
    if(head == NULL) {
        return NULL;
    }

    // If head node has the value
    if(head->data == value) {
        Node* temp = head;
        head = head->next;
        delete temp;
        return head;
    }

    // Search for the node
    Node* temp = head;
    while(temp->next != NULL && temp->next->data != value) {
        temp = temp->next;
    }

    // If value not found
    if(temp->next == NULL) {
        cout << "Value not found\n";
        return head;
    }

    // Delete node
```

```

Node* nodeToDelete = temp->next;
temp->next = temp->next->next;
delete nodeToDelete;

return head;
}

// Usage
Node* head = new Node(10);
head->next = new Node(20);
head->next->next = new Node(30);

head = deleteByValue(head, 20); // Delete node with value 20
// List: 10 -> 30 -> NULL

```

4 Traversal & Display - O(n)

```

void display(Node* head) {
    if(head == NULL) {
        cout << "List is empty\n";
        return;
    }

    Node* temp = head;
    while(temp != NULL) {
        cout << temp->data << " -> ";
        temp = temp->next;
    }
    cout << "NULL\n";
}

// Recursive display

```

```

void displayRecursive(Node* head) {
    if(head == NULL) {
        cout << "NULL\n";
        return;
    }
    cout << head->data << " -> ";
    displayRecursive(head->next);
}

// Usage
Node* head = new Node(10);
head->next = new Node(20);
head->next->next = new Node(30);

display(head); // 10 -> 20 -> 30 -> NULL

```

5 Search - O(n)

```

bool search(Node* head, int key) {
    Node* temp = head;

    while(temp != NULL) {
        if(temp->data == key) {
            return true;
        }
        temp = temp->next;
    }

    return false;
}

// Get position of element

```

```

int getPosition(Node* head, int key) {
    Node* temp = head;
    int pos = 0;

    while(temp != NULL) {
        if(temp->data == key) {
            return pos;
        }
        temp = temp->next;
        pos++;
    }

    return -1; // Not found
}

// Usage
Node* head = new Node(10);
head->next = new Node(20);
head->next->next = new Node(30);

cout << search(head, 20); // 1 (true)
cout << search(head, 40); // 0 (false)

cout << getPosition(head, 20); // 1
cout << getPosition(head, 40); // -1

```

6 Size & Access

```

// Get length of linked list
int getLength(Node* head) {
    int count = 0;
    Node* temp = head;

```

```

    while(temp != NULL) {
        count++;
        temp = temp->next;
    }

    return count;
}

// Get nth node data
int getNthNode(Node* head, int n) {
    Node* temp = head;

    for(int i = 0; i < n && temp != NULL; i++) {
        temp = temp->next;
    }

    if(temp == NULL) {
        cout << "Invalid position\n";
        return -1;
    }

    return temp->data;
}

// Get middle element
int getMiddle(Node* head) {
    Node* slow = head;
    Node* fast = head;

    while(fast != NULL && fast->next != NULL) {
        slow = slow->next;
        fast = fast->next->next;
    }
}

```

```

        return slow->data;
    }

    // Usage
    Node* head = new Node(10);
    head->next = new Node(20);
    head->next->next = new Node(30);
    head->next->next->next = new Node(40);

    cout << "Length: " << getLength(head); // 4
    cout << "3rd node: " << getNthNode(head, 2); // 30
    cout << "Middle: " << getMiddle(head); // 30

```

7 Reverse Linked List - O(n)

Iterative Approach

```

Node* reverseIterative(Node* head) {
    Node* prev = NULL;
    Node* current = head;
    Node* next = NULL;

    while(current != NULL) {
        next = current->next; // Store next
        current->next = prev; // Reverse link
        prev = current;       // Move prev forward
        current = next;       // Move current forward
    }

    return prev; // New head
}

```



```
// Usage
Node* head = new Node(10);
head->next = new Node(20);
head->next->next = new Node(30);
// Before: 10 -> 20 -> 30 -> NULL

head = reverseIterative(head);
// After: 30 -> 20 -> 10 -> NULL
```

Visual Step-by-Step:

```
Initial: NULL <- prev  current -> [10|•] -> [20|•] -> [30|NULL]
                                     next

Step 1: NULL <- [10|NULL]  prev  current -> [20|•] -> [30|NULL]
                                     next

Step 2: NULL <- [10|•] <- [20|NULL]  prev  current -> [30|NULL]
                                     next

Step 3: NULL <- [10|•] <- [20|•] <- [30|NULL]  prev
current=NULL
                                     next

Return prev as new head
```

Recursive Approach

```
Node* reverseRecursive(Node* head) {
    // Base case
    if(head == NULL || head->next == NULL) {
        return head;
    }
```

```

    }

    // Recursive call
    Node* newHead = reverseRecursive(head->next);

    // Reverse the link
    head->next->next = head;
    head->next = NULL;

    return newHead;
}

// Usage
Node* head = new Node(10);
head->next = new Node(20);
head->next->next = new Node(30);

head = reverseRecursive(head);
// Result: 30 -> 20 -> 10 -> NULL

```

8 Common Utility Operations

a) Count Occurrences

```

int countOccurrences(Node* head, int key) {
    int count = 0;
    Node* temp = head;

    while(temp != NULL) {
        if(temp->data == key) {
            count++;
        }
    }
}

```

```
        temp = temp->next;
    }

    return count;
}
```

b) Remove Duplicates (Sorted List)

```
void removeDuplicates(Node* head) {
    if(head == NULL) return;

    Node* current = head;

    while(current->next != NULL) {
        if(current->data == current->next->data) {
            Node* temp = current->next;
            current->next = current->next->next;
            delete temp;
        } else {
            current = current->next;
        }
    }
}

// Usage
// List: 10 -> 10 -> 20 -> 30 -> 30 -> NULL
removeDuplicates(head);
// Result: 10 -> 20 -> 30 -> NULL
```

c) Detect Cycle (Floyd's Algorithm)

```
bool hasCycle(Node* head) {  
    if(head == NULL) return false;  
  
    Node* slow = head;  
    Node* fast = head;  
  
    while(fast != NULL && fast->next != NULL) {  
        slow = slow->next;  
        fast = fast->next->next;  
  
        if(slow == fast) {  
            return true; // Cycle detected  
        }  
    }  
  
    return false; // No cycle  
}
```

d) Find Nth Node from End

```
int getNthFromEnd(Node* head, int n) {  
    Node* first = head;  
    Node* second = head;  
  
    // Move first pointer n steps ahead  
    for(int i = 0; i < n; i++) {  
        if(first == NULL) {  
            return -1; // n is greater than length  
        }  
        first = first->next;  
    }
```

```

    }

    // Move both pointers until first reaches end
    while(first != NULL) {
        first = first->next;
        second = second->next;
    }

    return second->data;
}

// Usage
// List: 10 -> 20 -> 30 -> 40 -> 50 -> NULL
cout << getNthFromEnd(head, 2); // 40 (2nd from end)

```

e) Merge Two Sorted Lists

```

Node* mergeSortedLists(Node* l1, Node* l2) {
    if(l1 == NULL) return l2;
    if(l2 == NULL) return l1;

    Node* result = NULL;

    if(l1->data <= l2->data) {
        result = l1;
        result->next = mergeSortedLists(l1->next, l2);
    } else {
        result = l2;
        result->next = mergeSortedLists(l1, l2->next);
    }

    return result;
}

```

```

}

// Usage
// List1: 10 -> 30 -> 50 -> NULL
// List2: 20 -> 40 -> 60 -> NULL
Node* merged = mergeSortedLists(list1, list2);
// Result: 10 -> 20 -> 30 -> 40 -> 50 -> 60 -> NULL

```

f) Delete Entire List

```

void deleteList(Node* &head) {
    Node* current = head;
    Node* next = NULL;

    while(current != NULL) {
        next = current->next;
        delete current;
        current = next;
    }

    head = NULL;
}

```

Time Complexity Summary (Singly Linked List)

Operation	Time Complexity	Space
Insert at beginning	O(1)	O(1)

Operation	Time Complexity	Space
Insert at end	$O(n)$	$O(1)$
Insert at position	$O(n)$	$O(1)$
Delete from beginning	$O(1)$	$O(1)$
Delete from end	$O(n)$	$O(1)$
Delete at position	$O(n)$	$O(1)$
Search	$O(n)$	$O(1)$
Access nth element	$O(n)$	$O(1)$
Reverse	$O(n)$	$O(1)$ iterative, $O(n)$ recursive
Length	$O(n)$	$O(1)$
Detect cycle	$O(n)$	$O(1)$

Shall I continue with **STL list<> (Doubly Linked List)** implementation next? 🚀

08/01/2026, 11:35:22

LINKED LIST (Continued)

APPROACH 2: STL list<> (Doubly Linked List)

What is STL list<>?

STL list C++ ka built-in **doubly linked list** implementation hai jo `<list>` header me available hai. Ye internally har node me **prev** aur **next** pointers rakhta hai.

Doubly Linked List Structure:

```
NULL <- [Prev|Data|Next] <-> [Prev|Data|Next] <->
[Prev|Data|Next] -> NULL
```

Advantages over Singly Linked List:

- Dono directions me traverse kar sakte ho (forward & backward)
- Delete operation zyada efficient hai
- Reverse traversal easy hai

Disadvantages:

- Extra memory (prev pointer ke liye)
- Slightly complex maintenance



Complete STL list<> Operations

1 Declaration & Initialization

```
#include <list>
#include <iostream>
using namespace std;

// Method 1: Empty list
list<int> l;

// Method 2: With size (default value 0)
list<int> l(5); // {0, 0, 0, 0, 0}

// Method 3: With size + value
list<int> l(5, 10); // {10, 10, 10, 10, 10}

// Method 4: Initialize with values
list<int> l = {10, 20, 30, 40, 50};

// Method 5: Copy from another list
list<int> l1 = {1, 2, 3};
list<int> l2(l1); // Copy constructor
list<int> l3 = l1; // Copy assignment

// Method 6: From array
int arr[] = {10, 20, 30};
list<int> l(arr, arr + 3);

// Method 7: From vector
vector<int> v = {10, 20, 30};
list<int> l(v.begin(), v.end());
```



```
// Method 8: Using initializer list
list<int> l({10, 20, 30});

// Different data types
list<string> names = {"Alice", "Bob", "Charlie"};
list<double> prices = {10.5, 20.3, 15.8};
list<pair<int, int>> pairs = {{1,2}, {3,4}};
```

Example:

```
#include <iostream>
#include <list>
using namespace std;

int main() {
    list<int> numbers = {10, 20, 30, 40, 50};

    cout << "Size: " << numbers.size() << endl;           // 5
    cout << "First: " << numbers.front() << endl;          // 10
    cout << "Last: " << numbers.back() << endl;           // 50

    return 0;
}
```

2 Insertion Operations

a) Insert at Beginning - O(1)

```
list<int> l = {20, 30, 40};

// push_front() - adds at beginning
l.push_front(10);
```

```
// Result: {10, 20, 30, 40}

l.push_front(5);
// Result: {5, 10, 20, 30, 40}

// emplace_front() - constructs in place (faster)
l.emplace_front(0);
// Result: {0, 5, 10, 20, 30, 40}
```

Visual:

```
Before: [20|•] <-> [30|•] <-> [40|NULL]
        ↑
        head

push_front(10):
After: [10|•] <-> [20|•] <-> [30|•] <-> [40|NULL]
        ↑
        head
```

b) Insert at End - O(1)

```
list<int> l = {10, 20, 30};

// push_back() - adds at end
l.push_back(40);
// Result: {10, 20, 30, 40}

l.push_back(50);
// Result: {10, 20, 30, 40, 50}

// emplace_back() - constructs in place
```

```
l.emplace_back(60);  
// Result: {10, 20, 30, 40, 50, 60}
```

Note: Unlike singly linked list, `push_back()` is **O(1)** in doubly linked list because we maintain a tail pointer.

c) Insert at Position - O(n)

```
list<int> l = {10, 20, 40, 50};  
  
// insert(iterator, value)  
auto it = l.begin();  
advance(it, 2); // Move iterator to position 2  
l.insert(it, 30);  
// Result: {10, 20, 30, 40, 50}  
  
// Insert at beginning  
l.insert(l.begin(), 5);  
// Result: {5, 10, 20, 30, 40, 50}  
  
// Insert at end  
l.insert(l.end(), 60);  
// Result: {5, 10, 20, 30, 40, 50, 60}  
  
// Insert multiple elements  
auto it2 = l.begin();  
advance(it2, 2);  
l.insert(it2, 3, 99); // Insert 3 copies of 99  
// Result: {5, 10, 99, 99, 99, 20, 30, 40, 50, 60}  
  
// Insert from another container
```

```
list<int> l2 = {100, 200};  
l.insert(l.end(), l2.begin(), l2.end());
```

Example: Insert in middle

```
list<int> scores = {85, 90, 95};  
  
// Insert 88 after 85 (at position 1)  
auto it = scores.begin();  
it++; // Move to position 1  
scores.insert(it, 88);  
  
// Display  
for(int x : scores) {  
    cout << x << " ";  
}  
// Output: 85 88 90 95
```

3 Deletion Operations

a) Delete from Beginning - O(1)

```
list<int> l = {10, 20, 30, 40, 50};  
  
// pop_front() - removes first element  
l.pop_front();  
// Result: {20, 30, 40, 50}  
  
l.pop_front();  
// Result: {30, 40, 50}  
  
// Check before deleting
```

```
if(!l.empty()) {  
    l.pop_front();  
}  
  
// Result: {40, 50}
```

b) Delete from End - O(1)

```
list<int> l = {10, 20, 30, 40, 50};  
  
// pop_back() - removes last element  
l.pop_back();  
// Result: {10, 20, 30, 40}  
  
l.pop_back();  
// Result: {10, 20, 30}  
  
// Check before deleting  
if(!l.empty()) {  
    l.pop_back();  
}  
  
// Result: {10, 20}
```

Note: `pop_back()` is **O(1)** in doubly linked list (unlike singly linked list where it's O(n)).

c) Delete at Position - O(n)

```
list<int> l = {10, 20, 30, 40, 50};  
  
// erase(iterator) - delete single element
```

```

auto it = l.begin();
advance(it, 2); // Move to position 2
l.erase(it);    // Delete 30
// Result: {10, 20, 40, 50}

// Delete first element
l.erase(l.begin());
// Result: {20, 40, 50}

// Delete range of elements
list<int> l2 = {10, 20, 30, 40, 50};
auto start = l2.begin();
auto end = l2.begin();
advance(start, 1); // Position 1
advance(end, 4);   // Position 4
l2.erase(start, end); // Delete [1, 4) - elements at index 1,2,3
// Result: {10, 50}

```

Example:

```

list<int> nums = {10, 20, 30, 40, 50};

// Delete element at position 2 (value 30)
auto it = nums.begin();
advance(it, 2);
nums.erase(it);

// Display
for(int x : nums) {
    cout << x << " ";
}
// Output: 10 20 40 50

```

d) Delete by Value - O(n)

```
list<int> l = {10, 20, 30, 20, 40};

// remove(value) - removes ALL occurrences
l.remove(20);
// Result: {10, 30, 40}

// Remove only first occurrence
list<int> l2 = {10, 20, 30, 20, 40};
auto it = find(l2.begin(), l2.end(), 20);
if(it != l2.end()) {
    l2.erase(it);
}
// Result: {10, 30, 20, 40} (only first 20 removed)
```

e) Delete with Condition - O(n)

```
list<int> l = {10, 15, 20, 25, 30, 35};

// remove_if() - remove elements matching condition
l.remove_if([](int x) { return x % 2 != 0; }); // Remove odd
// Result: {10, 20, 30}

// Remove elements greater than 25
list<int> l2 = {10, 20, 30, 40, 50};
l2.remove_if([](int x) { return x > 25; });
// Result: {10, 20}

// Remove negative numbers
list<int> l3 = {-5, 10, -3, 20, -1};
```



```
l3.remove_if([](int x) { return x < 0; });  
// Result: {10, 20}
```

f) Clear All Elements - $O(n)$

```
list<int> l = {10, 20, 30, 40, 50};  
  
l.clear(); // Removes all elements  
// Result: {} (empty list)  
  
cout << "Size: " << l.size(); // 0  
cout << "Empty: " << l.empty(); // 1 (true)
```

4 Access Elements

```
list<int> l = {10, 20, 30, 40, 50};  
  
// ❌ l[0] - NOT POSSIBLE (no random access)  
// ❌ l.at(2) - NOT POSSIBLE  
  
// ✅ front() - first element -  $O(1)$   
cout << l.front(); // 10  
  
// ✅ back() - last element -  $O(1)$   
cout << l.back(); // 50  
  
// ✅ Access using iterator -  $O(n)$   
auto it = l.begin();  
advance(it, 2); // Move to position 2
```

```
cout << *it;      // 30

//  Access nth element (manual)
int getNth(list<int>& l, int n) {
    auto it = l.begin();
    advance(it, n);
    return *it;
}

cout << getNth(l, 3); // 40
```

Important: List does NOT support random access like vector or array. You CANNOT use `l[i]`.

5 Update Elements

```
list<int> l = {10, 20, 30, 40, 50};

// Update first element
l.front() = 15;
// Result: {15, 20, 30, 40, 50}

// Update last element
l.back() = 55;
// Result: {15, 20, 30, 40, 55}

// Update using iterator
auto it = l.begin();
advance(it, 2); // Position 2
*it = 35;
// Result: {15, 20, 35, 40, 55}
```

```
// Update all elements (multiply by 2)
for(auto it = l.begin(); it != l.end(); it++) {
    *it = (*it) * 2;
}
// Result: {30, 40, 70, 80, 110}
```

6 Traversal - O(n)

```
list<int> l = {10, 20, 30, 40, 50};

// Method 1: Range-based for loop (Best for reading)
for(int x : l) {
    cout << x << " ";
}
// Output: 10 20 30 40 50

// Method 2: Range-based with reference (for modification)
for(int &x : l) {
    x = x * 2; // Modifies original list
}
// Result: {20, 40, 60, 80, 100}

// Method 3: Using iterators (forward)
for(auto it = l.begin(); it != l.end(); it++) {
    cout << *it << " ";
}
// Output: 20 40 60 80 100

// Method 4: Using reverse iterators (backward)
for(auto it = l.rbegin(); it != l.rend(); it++) {
    cout << *it << " ";
}
```

```
// Output: 100 80 60 40 20
```

```
// Method 5: Using auto (C++11)
```

```
for(auto it = l.begin(); it != l.end(); ++it) {  
    cout << *it << " ";  
}
```

Example: Print even numbers only

```
list<int> nums = {10, 15, 20, 25, 30, 35};
```

```
cout << "Even numbers: ";
```

```
for(int x : nums) {  
    if(x % 2 == 0) {  
        cout << x << " ";  
    }  
}
```

```
// Output: Even numbers: 10 20 30
```

7 Search - O(n)

```
#include <algorithm>
```

```
list<int> l = {10, 20, 30, 40, 50};
```

```
// find() - returns iterator to first occurrence
```

```
auto it = find(l.begin(), l.end(), 30);
```

```
if(it != l.end()) {
```

```
    cout << "Found: " << *it << endl; // Found: 30
```

```
} else {
```

```
    cout << "Not found" << endl;
```

```
}
```

```

// Check if element exists
if(find(l.begin(), l.end(), 30) != l.end()) {
    cout << "30 exists";
}

// count() - count occurrences
list<int> l2 = {10, 20, 10, 30, 10};
int cnt = count(l2.begin(), l2.end(), 10);
cout << "Count of 10: " << cnt; // 3

// find_if() - find with condition
auto it2 = find_if(l.begin(), l.end(), [](int x) { return x > 25;
});
if(it2 != l.end()) {
    cout << "First element > 25: " << *it2; // 30
}

// Get position of element
int getPosition(list<int>& l, int value) {
    auto it = find(l.begin(), l.end(), value);
    if(it != l.end()) {
        return distance(l.begin(), it);
    }
    return -1;
}

cout << getPosition(l, 30); // 2

```

8 Size & Empty Check

```

list<int> l = {10, 20, 30};

// size() - number of elements - O(1)
cout << "Size: " << l.size(); // 3

// empty() - check if empty - O(1)
if(l.empty()) {
    cout << "List is empty";
} else {
    cout << "List is not empty"; // This prints
}

// max_size() - maximum possible size
cout << l.max_size(); // Very large number

// After clearing
l.clear();
cout << "Size: " << l.size(); // 0
cout << "Empty: " << l.empty(); // 1 (true)

```

9 Reverse - O(n)

```

list<int> l = {10, 20, 30, 40, 50};

// reverse() - reverses the list in-place
l.reverse();

for(int x : l) {
    cout << x << " ";
}

// Output: 50 40 30 20 10

```

Visual:

Before: [10] <-> [20] <-> [30] <-> [40] <-> [50]

After `reverse()`:

[50] <-> [40] <-> [30] <-> [20] <-> [10]

10 Sort - $O(n \log n)$

```
list<int> l = {50, 20, 40, 10, 30};

// sort() - sorts in ascending order
l.sort();

for(int x : l) {
    cout << x << " ";
}
// Output: 10 20 30 40 50

// sort() with comparator - descending order
l.sort(greater<int>());

for(int x : l) {
    cout << x << " ";
}
// Output: 50 40 30 20 10

// Custom comparator
list<pair<int, int>> pairs = {{1, 50}, {2, 20}, {3, 40}};

// Sort by second element
pairs.sort([](pair<int,int> a, pair<int,int> b) {
```

```

        return a.second < b.second;
    });
    // Result: {{2,20}, {3,40}, {1,50}}

    // Sort strings by length
    list<string> words = {"apple", "hi", "banana", "car"};
    words.sort([](string a, string b) {
        return a.length() < b.length();
    });
    // Result: {"hi", "car", "apple", "banana"}

```

Note: Use `list::sort()` instead of `std::sort()` for lists because lists don't support random access iterators.

1 1 Unique (Remove Consecutive Duplicates) - O(n)

```

list<int> l = {10, 10, 20, 20, 20, 30, 30, 40};

// unique() - removes consecutive duplicates
l.unique();

for(int x : l) {
    cout << x << " ";
}
// Output: 10 20 30 40

// With comparator (remove if difference < 5)
list<int> l2 = {1, 2, 5, 6, 10, 11, 15};
l2.unique([](int a, int b) { return abs(a - b) < 5; });
// Result: {1, 5, 10, 15}

```


Important: `unique()` only removes CONSECUTIVE duplicates. To remove all duplicates, first sort then unique:

```
list<int> l = {10, 20, 10, 30, 20};

l.sort();    // {10, 10, 20, 20, 30}
l.unique();  // {10, 20, 30}
```

1 2 Merge Two Sorted Lists - $O(n + m)$

```
list<int> l1 = {10, 30, 50};
list<int> l2 = {20, 40, 60};

// merge() - merges l2 into l1 (both must be sorted)
l1.merge(l2);

cout << "L1: ";
for(int x : l1) {
    cout << x << " ";
}
// Output: L1: 10 20 30 40 50 60

cout << "\nL2: ";
for(int x : l2) {
    cout << x << " ";
}
// Output: L2: (empty - elements moved to l1)

// Merge with custom comparator
list<int> l3 = {50, 30, 10}; // Descending
list<int> l4 = {60, 40, 20}; // Descending
```

```
l3.merge(l4, greater<int>());  
// Result: {60, 50, 40, 30, 20, 10}
```

13 Splice (Transfer Elements) - O(1) or O(n)

```
list<int> l1 = {10, 20, 30};  
list<int> l2 = {40, 50, 60};  
  
// splice(position, source_list) - move entire list  
auto it = l1.end();  
l1.splice(it, l2);  
// L1: {10, 20, 30, 40, 50, 60}  
// L2: {} (empty)  
  
// splice(position, source_list, iterator) - move single element  
list<int> l3 = {10, 20, 30};  
list<int> l4 = {40, 50, 60};  
  
auto it2 = l4.begin();  
advance(it2, 1); // Points to 50  
l3.splice(l3.end(), l4, it2);  
// L3: {10, 20, 30, 50}  
// L4: {40, 60}  
  
// splice(position, source_list, start, end) - move range  
list<int> l5 = {10, 20};  
list<int> l6 = {30, 40, 50, 60};  
  
auto start = l6.begin();  
auto end = l6.begin();  
advance(start, 1); // Points to 40  
advance(end, 3);   // Points to 60
```

```
l5.splice(l5.end(), l6, start, end);  
// L5: {10, 20, 40, 50}  
// L6: {30, 60}
```

14 Resize - O(n)

```
list<int> l = {10, 20, 30};  
  
// Increase size (new elements = 0)  
l.resize(5);  
// Result: {10, 20, 30, 0, 0}  
  
// Increase size with value  
list<int> l2 = {10, 20, 30};  
l2.resize(6, 99);  
// Result: {10, 20, 30, 99, 99, 99}  
  
// Decrease size (truncate)  
list<int> l3 = {10, 20, 30, 40, 50};  
l3.resize(3);  
// Result: {10, 20, 30}
```

15 Swap Lists - O(1)

```
list<int> l1 = {10, 20, 30};  
list<int> l2 = {40, 50};  
  
// swap() - exchanges contents
```

```
l1.swap(l2);

cout << "L1: ";
for(int x : l1) cout << x << " ";
// Output: L1: 40 50

cout << "\nL2: ";
for(int x : l2) cout << x << " ";
// Output: L2: 10 20 30
```

16 Assign - O(n)

```
list<int> l;

// assign(n, value) - assign n copies of value
l.assign(5, 10);
// Result: {10, 10, 10, 10, 10}

// assign(start, end) - assign from range
vector<int> v = {20, 30, 40};
l.assign(v.begin(), v.end());
// Result: {20, 30, 40}

// assign from initializer list
l.assign({50, 60, 70});
// Result: {50, 60, 70}
```

Time Complexity Summary (Doubly Linked List)

Operation	Time Complexity	Explanation
push_front()	$O(1)$	Direct insertion at head
push_back()	$O(1)$	Direct insertion at tail
pop_front()	$O(1)$	Direct deletion from head
pop_back()	$O(1)$	Direct deletion from tail
insert()	$O(n)$	Need to traverse to position
erase()	$O(n)$	Need to find position
front()	$O(1)$	Direct access
back()	$O(1)$	Direct access
size()	$O(1)$	Maintained counter
reverse()	$O(n)$	Reverse all links
sort()	$O(n \log n)$	Merge sort
unique()	$O(n)$	Single pass
merge()	$O(n + m)$	Merge two sorted lists
splice()	$O(1)$ or $O(n)$	Depends on operation
find()	$O(n)$	Linear search

Interview Problems with STL list<>

Problem 1: Remove Duplicates (Unsorted)

```
void removeDuplicates(list<int>& l) {
    l.sort();           //  $O(n \log n)$ 
    l.unique();         //  $O(n)$ 
}
```

```
list<int> nums = {10, 20, 10, 30, 20, 40};
removeDuplicates(nums);
// Result: {10, 20, 30, 40}

for(int x : nums) {
    cout << x << " ";
}
```

Problem 2: Palindrome Check

```
bool isPalindrome(list<int> l) {
    list<int> reversed = l;
    reversed.reverse();

    return l == reversed;
}

list<int> l1 = {1, 2, 3, 2, 1};
list<int> l2 = {1, 2, 3, 4, 5};

cout << isPalindrome(l1); // 1 (true)
cout << isPalindrome(l2); // 0 (false)
```

Problem 3: Find Middle Element

```
int findMiddle(list<int>& l) {
    auto slow = l.begin();
    auto fast = l.begin();
```

```

    while(fast != l.end()) {
        advance(fast, 1);
        if(fast != l.end()) {
            advance(fast, 1);
            advance(slow, 1);
        }
    }

    return *slow;
}

list<int> nums = {10, 20, 30, 40, 50};
cout << findMiddle(nums); // 30

```

Problem 4: Partition List (Even/Odd)

```

void partitionEvenOdd(list<int>& l) {
    list<int> even, odd;

    for(int x : l) {
        if(x % 2 == 0) {
            even.push_back(x);
        } else {
            odd.push_back(x);
        }
    }

    l = even;
    l.splice(l.end(), odd);
}

list<int> nums = {1, 2, 3, 4, 5, 6};

```

```
partitionEvenOdd(nums);  
// Result: {2, 4, 6, 1, 3, 5}  
  
for(int x : nums) {  
    cout << x << " ";  
}
```

Problem 5: Rotate List

```
void rotateLeft(list<int>& l, int k) {  
    if(l.empty() || k == 0) return;  
  
    k = k % l.size(); // Handle k > size  
  
    auto it = l.begin();  
    advance(it, k);  
  
    l.splice(l.end(), l, l.begin(), it);  
}  
  
list<int> nums = {10, 20, 30, 40, 50};  
rotateLeft(nums, 2);  
// Result: {30, 40, 50, 10, 20}  
  
for(int x : nums) {  
    cout << x << " ";  
}
```

Problem 6: Intersection of Two Lists




```

list<int> intersection(list<int> l1, list<int> l2) {
    l1.sort();
    l2.sort();

    list<int> result;

    auto it1 = l1.begin();
    auto it2 = l2.begin();

    while(it1 != l1.end() && it2 != l2.end()) {
        if(*it1 == *it2) {
            result.push_back(*it1);
            it1++;
            it2++;
        } else if(*it1 < *it2) {
            it1++;
        } else {
            it2++;
        }
    }

    return result;
}

list<int> l1 = {1, 3, 4, 5, 7};
list<int> l2 = {2, 3, 5, 6};

list<int> result = intersection(l1, l2);
// Result: {3, 5}

for(int x : result) {
    cout << x << " ";
}

```

Problem 7: Flatten Nested List

```
void flatten(list<list<int>>& nested, list<int>& result) {
    for(auto& sublist : nested) {
        for(int x : sublist) {
            result.push_back(x);
        }
    }
}

list<list<int>> nested = {{1, 2}, {3, 4, 5}, {6}};
list<int> result;

flatten(nested, result);
// Result: {1, 2, 3, 4, 5, 6}

for(int x : result) {
    cout << x << " ";
}
```

Shall I continue with **Stack** next? 🚀

08/01/2026, 11:37:41

5 STACK

What is Stack?

Stack ek **LIFO (Last In First Out)** data structure hai jisme jo element last me insert hota hai, wo pehle remove hota hai.

Real Life Example:

- **Books ka pile** - Upar wali book pehle uthate ho
- **Plates ki stack** - Sabse upar wali plate pehle lete ho
- **Browser back button** - Last visited page pehle aata hai
- **Undo operation** - Last change pehle undo hota hai

Visual:

```
|      |  
|  30  |  <- Top (last inserted, first to remove)  
|  20  |  
|  10  |  <- Bottom (first inserted)  
-----
```

Key Operations:

- **Push** - Element add karo (top pe)
- **Pop** - Element remove karo (top se)
- **Top** - Top element dekho (without removing)

How It Works?

Empty Stack:

| |

Push 10:

| 10 | <- Top

Push 20:

| 20 | <- Top

| 10 |

Push 30:

| 30 | <- Top

| 20 |

| 10 |

Pop:

| 20 | <- Top (30 removed)

| 10 |

Pop:

| 10 | <- Top (20 removed)

STL stack<> Implementation

1 Declaration & Initialization

```
#include <stack>
#include <iostream>
using namespace std;
```

```
// Method 1: Empty stack
stack<int> st;

// Method 2: Stack of different types
stack<int> intStack;
stack<string> stringStack;
stack<char> charStack;
stack<double> doubleStack;

// Method 3: Stack of pairs
stack<pair<int, int>> pairStack;

// Method 4: Stack of vectors
stack<vector<int>> vectorStack;

// Note: You CANNOT initialize stack with values directly
// ❌ stack<int> st = {10, 20, 30}; // NOT POSSIBLE

// But you can push elements
stack<int> st;
st.push(10);
st.push(20);
st.push(30);
```

Example:

```
#include <iostream>
#include <stack>
using namespace std;

int main() {
    stack<int> st;

    st.push(10);
    st.push(20);
```

```
    st.push(30);

    cout << "Top: " << st.top() << endl;    // 30
    cout << "Size: " << st.size() << endl;    // 3

    return 0;
}
```

2 Push (Insert) - O(1)

```
stack<int> st;

// push() - adds element at top
st.push(10); // Stack: [10]
st.push(20); // Stack: [10, 20]
st.push(30); // Stack: [10, 20, 30]
st.push(40); // Stack: [10, 20, 30, 40]

cout << "Top: " << st.top(); // 40

// emplace() - constructs element in place (faster for complex
objects)
st.emplace(50); // Stack: [10, 20, 30, 40, 50]

// Push pairs
stack<pair<int, string>> st2;
st2.push({1, "Alice"});
st2.push({2, "Bob"});
st2.emplace(3, "Charlie"); // Better performance
```

Visual:

Initial: Empty

Push 10:

| 10 | <- Top

Push 20:

| 20 | <- Top

| 10 |

Push 30:

| 30 | <- Top

| 20 |

| 10 |

3 Pop (Delete) - O(1)

```
stack<int> st;
st.push(10);
st.push(20);
st.push(30);
// Stack: [10, 20, 30] (30 is top)

// pop() - removes top element
st.pop(); // Removes 30
// Stack: [10, 20] (20 is top)

cout << "Top after pop: " << st.top(); // 20

st.pop(); // Removes 20
// Stack: [10] (10 is top)

st.pop(); // Removes 10
```

```
// Stack: [] (empty)

// ⚠️ IMPORTANT: pop() does NOT return the value
// ❌ int x = st.pop(); // ERROR!

// ✅ Correct way to get and remove:
int x = st.top(); // Get value
st.pop();        // Then remove

// Always check before pop
if(!st.empty()) {
    st.pop();
}
```

Visual:

Before:

```
| 30 | <- Top
| 20 |
| 10 |
```

After `pop()`:

```
| 20 | <- Top
| 10 |
```

After `pop()`:

```
| 10 | <- Top
```

After `pop()`:

Empty

4 Top (Access) - $O(1)$


```

stack<int> st;
st.push(10);
st.push(20);
st.push(30);

// top() - returns reference to top element
cout << st.top(); // 30

// Modify top element
st.top() = 99;
cout << st.top(); // 99

// top() on empty stack is UNDEFINED BEHAVIOR
stack<int> empty_st;
// cout << empty_st.top(); // ❌ DANGEROUS!

// Safe access
if(!st.empty()) {
    cout << st.top(); // Safe
}

```

Example:

```

stack<string> names;
names.push("Alice");
names.push("Bob");
names.push("Charlie");

cout << "Last name: " << names.top() << endl; // Charlie

names.pop();
cout << "Last name now: " << names.top() << endl; // Bob

```

5 Size & Empty Check - O(1)

```
stack<int> st;

// empty() - returns true if stack is empty
cout << st.empty(); // 1 (true)

st.push(10);
st.push(20);

// size() - returns number of elements
cout << st.size(); // 2
cout << st.empty(); // 0 (false)

st.pop();
cout << st.size(); // 1

st.pop();
cout << st.empty(); // 1 (true)
cout << st.size(); // 0
```

Example:

```
stack<int> st;
st.push(10);
st.push(20);
st.push(30);

cout << "Stack size: " << st.size() << endl; // 3

while(!st.empty()) {
    cout << st.top() << " ";
    st.pop();
}

// Output: 30 20 10
```

```
cout << "\nStack size after: " << st.size() << endl; // 0
```

6 Traversal (Display All Elements)

Important: Stack doesn't support direct iteration. To traverse, you need to pop elements (which destroys the stack).

```
stack<int> st;
st.push(10);
st.push(20);
st.push(30);

// Method 1: Pop all elements (destroys stack)
cout << "Stack elements: ";
while(!st.empty()) {
    cout << st.top() << " ";
    st.pop();
}
// Output: 30 20 10
// Stack is now empty

// Method 2: Copy to temporary stack (preserves original)
void displayStack(stack<int> st) { // Pass by value (creates copy)
    while(!st.empty()) {
        cout << st.top() << " ";
        st.pop();
    }
    cout << endl;
}
```

```
stack<int> original;
original.push(10);
original.push(20);
original.push(30);

displayStack(original); // 30 20 10
cout << original.size(); // 3 (original preserved)

// Method 3: Copy to vector for traversal
stack<int> st2;
st2.push(10);
st2.push(20);
st2.push(30);

vector<int> v;
stack<int> temp = st2; // Copy

while(!temp.empty()) {
    v.push_back(temp.top());
    temp.pop();
}

// Display from vector (top to bottom)
for(int x : v) {
    cout << x << " "; // 30 20 10
}

// Display from vector (bottom to top)
reverse(v.begin(), v.end());
for(int x : v) {
    cout << x << " "; // 10 20 30
}
```

7 Swap Stacks - O(1)

```
stack<int> st1;
st1.push(10);
st1.push(20);

stack<int> st2;
st2.push(30);
st2.push(40);
st2.push(50);

cout << "st1 size: " << st1.size(); // 2
cout << "st2 size: " << st2.size(); // 3

// swap() - exchanges contents
st1.swap(st2);

cout << "After swap:" << endl;
cout << "st1 size: " << st1.size(); // 3
cout << "st2 size: " << st2.size(); // 2

cout << "st1 top: " << st1.top(); // 50
cout << "st2 top: " << st2.top(); // 20
```



Time Complexity Summary

Operation	Time Complexity	Space
push()	O(1)	O(1)
pop()	O(1)	O(1)
top()	O(1)	O(1)



Operation	Time Complexity	Space
size()	O(1)	O(1)
empty()	O(1)	O(1)
swap()	O(1)	O(1)

Space Complexity: O(n) where n = number of elements

Interview Problems with Stack

Problem 1: Reverse String Using Stack

```
string reverseString(string s) {
    stack<char> st;

    // Push all characters
    for(char c : s) {
        st.push(c);
    }

    // Pop all characters
    string result = "";
    while(!st.empty()) {
        result += st.top();
        st.pop();
    }

    return result;
}

cout << reverseString("hello"); // "olleh"
cout << reverseString("stack"); // "kcats"
```

How it works:

Input: "hello"

Push: h -> e -> l -> l -> o

Stack: [h, e, l, l, o] (o is top)

Pop: o -> l -> l -> e -> h

Output: "olleh"

Problem 2: Balanced Parentheses

```
bool isBalanced(string s) {
    stack<char> st;

    for(char c : s) {
        // Opening brackets - push
        if(c == '(' || c == '{' || c == '[') {
            st.push(c);
        }
        // Closing brackets - check match
        else if(c == ')' || c == '}' || c == ']') {
            if(st.empty()) return false;

            char top = st.top();
            if((c == ')' && top == '(') ||
               (c == '}' && top == '{') ||
               (c == ']' && top == '[')) {
                st.pop();
            } else {
                return false;
            }
        }
    }
}
```

```

    }
}

return st.empty(); // All brackets matched
}

cout << isBalanced("()[{}]"); // 1 (true)
cout << isBalanced("([{}])"); // 1 (true)
cout << isBalanced("()"); // 0 (false)
cout << isBalanced("([])"); // 0 (false)
cout << isBalanced("{[]}"); // 0 (false)

```

How it works:

Input: "([{}])"

Step 1: (-> push

Stack: [(

Step 2: [-> push

Stack: [(, [

Step 3: { -> push

Stack: [(, [, {

Step 4: } -> matches { -> pop

Stack: [(, [

Step 5:] -> matches [-> pop

Stack: [(

Step 6:) -> matches (-> pop

Stack: []

Empty stack = Balanced ✓

Problem 3: Next Greater Element

```
vector<int> nextGreater(vector<int> arr) {
    int n = arr.size();
    vector<int> result(n, -1);
    stack<int> st; // Store indices

    for(int i = 0; i < n; i++) {
        while(!st.empty() && arr[st.top()] < arr[i]) {
            result[st.top()] = arr[i];
            st.pop();
        }
        st.push(i);
    }

    return result;
}

vector<int> arr = {4, 5, 2, 10, 8};
vector<int> result = nextGreater(arr);

for(int x : result) {
    cout << x << " ";
}
// Output: 5 10 10 -1 -1
```

Explanation:

Array: [4, 5, 2, 10, 8]

For 4: Next greater = 5

For 5: Next greater = 10

For 2: Next greater = 10

For 10: Next greater = -1 (none)

For 8: Next greater = -1 (none)

Result: [5, 10, 10, -1, -1]

Problem 4: Valid Arithmetic Expression

```
int precedence(char op) {
    if(op == '+' || op == '-') return 1;
    if(op == '*' || op == '/') return 2;
    return 0;
}

bool isOperator(char c) {
    return c == '+' || c == '-' || c == '*' || c == '/';
}

string infixToPostfix(string s) {
    stack<char> st;
    string result = "";

    for(char c : s) {
        // Operand - add to result
        if(isalnum(c)) {
            result += c;
        }
        // Opening bracket - push
```

```

        else if(c == '(') {
            st.push(c);
        }
        // Closing bracket - pop until '('
        else if(c == ')') {
            while(!st.empty() && st.top() != '(') {
                result += st.top();
                st.pop();
            }
            if(!st.empty()) st.pop(); // Remove '('
        }
        // Operator
        else if(isOperator(c)) {
            while(!st.empty() && precedence(st.top()) >=
precedence(c)) {
                result += st.top();
                st.pop();
            }
            st.push(c);
        }
    }

    // Pop remaining operators
    while(!st.empty()) {
        result += st.top();
        st.pop();
    }

    return result;
}

cout << infixToPostfix("a+b*c"); // "abc*+"
cout << infixToPostfix("(a+b)*c"); // "ab+c*"
cout << infixToPostfix("a+b*(c-d)"); // "abcd- *+"

```

Problem 5: Stock Span Problem

```
vector<int> calculateSpan(vector<int> prices) {
    int n = prices.size();
    vector<int> span(n);
    stack<int> st; // Store indices

    for(int i = 0; i < n; i++) {
        // Pop elements smaller than current
        while(!st.empty() && prices[st.top()] <= prices[i]) {
            st.pop();
        }

        // Calculate span
        span[i] = st.empty() ? (i + 1) : (i - st.top());

        st.push(i);
    }

    return span;
}

vector<int> prices = {100, 80, 60, 70, 60, 75, 85};
vector<int> span = calculateSpan(prices);

for(int x : span) {
    cout << x << " ";
}

// Output: 1 1 1 2 1 4 6
```

Explanation:

Prices: [100, 80, 60, 70, 60, 75, 85]

Span: [1, 1, 1, 2, 1, 4, 6]

For 100: 1 day (no previous higher)

For 80: 1 day (100 is higher)

For 60: 1 day (80 is higher)

For 70: 2 days (60 < 70, but 80 > 70)

For 60: 1 day (70 is higher)

For 75: 4 days (60,70,60,75 all counted)

For 85: 6 days (all previous counted except 100)

Problem 6: Min Stack (Get Minimum in O(1))

```
class MinStack {
    stack<int> st;
    stack<int> minSt;

public:
    void push(int x) {
        st.push(x);

        if(minSt.empty() || x <= minSt.top()) {
            minSt.push(x);
        }
    }

    void pop() {
        if(st.top() == minSt.top()) {
            minSt.pop();
        }
        st.pop();
    }
}
```

```

int top() {
    return st.top();
}

int getMin() {
    return minSt.top();
}
};

// Usage
MinStack st;
st.push(5);
st.push(3);
st.push(7);
st.push(2);

cout << st.getMin(); // 2

st.pop();
cout << st.getMin(); // 3

st.pop();
cout << st.getMin(); // 3

```

Problem 7: Remove Adjacent Duplicates

```

string removeDuplicates(string s) {
    stack<char> st;

    for(char c : s) {
        if(!st.empty() && st.top() == c) {

```

```

        st.pop(); // Remove duplicate
    } else {
        st.push(c);
    }
}

// Build result from stack
string result = "";
while(!st.empty()) {
    result = st.top() + result;
    st.pop();
}

return result;
}

cout << removeDuplicates("abbaca"); // "ca"
cout << removeDuplicates("azxxzy"); // "ay"

```

How it works:

Input: "abbaca"

```

a -> push: [a]
b -> push: [a, b]
b -> matches top -> pop: [a]
a -> matches top -> pop: []
c -> push: [c]
a -> push: [c, a]

```

Result: "ca"

Problem 8: Implement Queue Using Two Stacks

```
class QueueUsingStacks {
    stack<int> s1, s2;

public:
    void enqueue(int x) {
        s1.push(x);
    }

    int dequeue() {
        if(s2.empty()) {
            while(!s1.empty()) {
                s2.push(s1.top());
                s1.pop();
            }
        }

        int front = s2.top();
        s2.pop();
        return front;
    }

    bool empty() {
        return s1.empty() && s2.empty();
    }
};

// Usage
QueueUsingStacks q;
q.enqueue(10);
q.enqueue(20);
q.enqueue(30);
```



```
cout << q.dequeue(); // 10
cout << q.dequeue(); // 20
```

Problem 9: Evaluate Postfix Expression

```
int evaluatePostfix(string s) {
    stack<int> st;

    for(char c : s) {
        if(isdigit(c)) {
            st.push(c - '0'); // Convert char to int
        } else {
            int val2 = st.top(); st.pop();
            int val1 = st.top(); st.pop();

            switch(c) {
                case '+': st.push(val1 + val2); break;
                case '-': st.push(val1 - val2); break;
                case '*': st.push(val1 * val2); break;
                case '/': st.push(val1 / val2); break;
            }
        }
    }

    return st.top();
}

cout << evaluatePostfix("231*+9-"); // -4
// Explanation: 2 + (3 * 1) - 9 = 2 + 3 - 9 = -4
```

Problem 10: Check Redundant Parentheses

```
bool hasRedundant(string s) {
    stack<char> st;

    for(char c : s) {
        if(c == ')') {
            char top = st.top();
            st.pop();

            bool hasOperator = false;

            while(top != '(') {
                if(top == '+' || top == '-' || top == '*' || top
== '/') {
                    hasOperator = true;
                }
                top = st.top();
                st.pop();
            }

            if(!hasOperator) {
                return true; // Redundant
            }
        } else {
            st.push(c);
        }
    }

    return false;
}

cout << hasRedundant("((a+b))"); // 1 (redundant)
cout << hasRedundant("(a+(b*c))"); // 0 (not redundant)
cout << hasRedundant("(a+b)"); // 0 (not redundant)
```

⚠ Common Mistakes & Tips

❌ Mistake 1: Using top() on empty stack

```
stack<int> st;  
cout << st.top(); // ❌ Undefined behavior (crash)
```

✅ Solution:

```
if(!st.empty()) {  
    cout << st.top(); // Safe  
}
```

❌ Mistake 2: Thinking pop() returns value

```
stack<int> st;  
st.push(10);  
int x = st.pop(); // ❌ ERROR! pop() returns void
```

✅ Solution:

```
int x = st.top(); // Get value first  
st.pop();        // Then remove
```

❌ Mistake 3: Not preserving stack while traversing

```
stack<int> st;
st.push(10);
st.push(20);

while(!st.empty()) {
    cout << st.top();
    st.pop(); // ❌ Stack destroyed
}
// Now st is empty!
```

✅ Solution:

```
stack<int> temp = st; // Create copy
while(!temp.empty()) {
    cout << temp.top();
    temp.pop();
}
// Original st preserved
```

🚀 Pro Tips

1. Always check empty before top/pop:

```
if(!st.empty()) {
    int x = st.top();
    st.pop();
}
```

2. Use stack for:

- Parentheses matching

- Expression evaluation
- Undo/Redo operations
- DFS traversal
- Backtracking problems

3. **Stack vs Vector:**

- Use stack when you ONLY need LIFO operations
- Use vector when you need random access

4. **Memory efficiency:**

```
stack<int> st;  
st.push(1000000); // Uses only space for elements  
// No pre-allocation like vector
```

When to Use Stack?

Use Stack When:

- Need LIFO behavior
- Reversing data
- Checking balanced parentheses
- Expression evaluation (infix/postfix)
- DFS algorithm
- Undo/Redo functionality
- Function call stack simulation
- Backtracking problems

Don't Use Stack When:

- Need random access (use vector/array)
 - Need FIFO behavior (use queue)
 - Need to access middle elements frequently
-

Ready for **Queue & Deque** next? 🚀

08/01/2026, 11:58:18

QUEUE & DEQUE

PART A: QUEUE

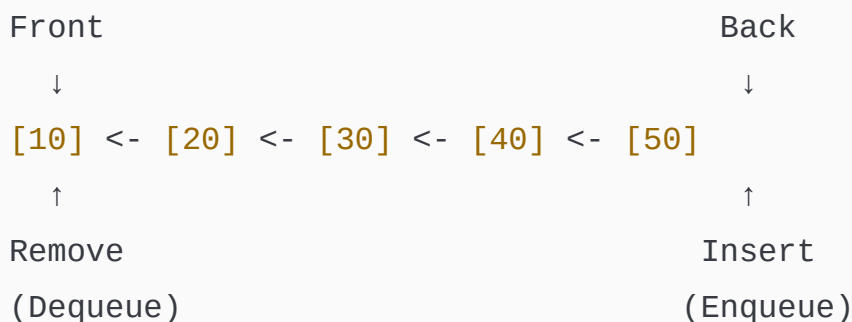
What is Queue?

Queue ek **FIFO (First In First Out)** data structure hai jisme jo element pehle insert hota hai, wo pehle remove hota hai.

Real Life Examples:

- **Movie ticket counter** - Jo pehle line me aaya, wo pehle ticket lega
- **Printer queue** - Jo document pehle print ke liye gaya, wo pehle print hoga
- **Customer service** - Jo customer pehle aaya, uski request pehle process hogi
- **Restaurant order** - Jo order pehle place hua, wo pehle ready hoga

Visual:



Key Operations:

- **Enqueue (push)** - Element add karo (back/rear pe)
- **Dequeue (pop)** - Element remove karo (front se)
- **Front** - Front element dekho (without removing)
- **Back** - Back element dekho (without removing)

How Queue Works?

Empty Queue:

Front -> | | <- Back

Enqueue 10:

Front -> | 10 | <- Back

Enqueue 20:

Front -> | 10 | 20 | <- Back

Enqueue 30:

Front -> | 10 | 20 | 30 | <- Back

Dequeue:

Front -> | 20 | 30 | <- Back (10 removed)

Dequeue:

Front -> | 30 | <- Back (20 removed)

STL queue<> Implementation

1 Declaration & Initialization


```
#include <queue>
#include <iostream>
using namespace std;

// Method 1: Empty queue
queue<int> q;

// Method 2: Queue of different types
queue<int> intQueue;
queue<string> stringQueue;
queue<char> charQueue;
queue<double> doubleQueue;

// Method 3: Queue of pairs
queue<pair<int, int>> pairQueue;

// Method 4: Queue of vectors
queue<vector<int>> vectorQueue;

// Note: You CANNOT initialize queue with values directly
// ❌ queue<int> q = {10, 20, 30}; // NOT POSSIBLE

// But you can push elements
queue<int> q;
q.push(10);
q.push(20);
q.push(30);

// Method 5: Copy from another queue
queue<int> q1;
q1.push(10);
q1.push(20);

queue<int> q2 = q1; // Copy constructor
```

Example:

```
#include <iostream>
#include <queue>
using namespace std;

int main() {
    queue<int> q;

    q.push(10);
    q.push(20);
    q.push(30);

    cout << "Front: " << q.front() << endl; // 10
    cout << "Back: " << q.back() << endl;    // 30
    cout << "Size: " << q.size() << endl;     // 3

    return 0;
}
```

2 Enqueue (Insert at Back) - O(1)

```
queue<int> q;

// push() - adds element at back/rear
q.push(10); // Queue: [10]
q.push(20); // Queue: [10, 20]
q.push(30); // Queue: [10, 20, 30]
q.push(40); // Queue: [10, 20, 30, 40]

cout << "Front: " << q.front(); // 10
cout << "Back: " << q.back();   // 40
```

```
// emplace() - constructs element in place (faster for complex objects)
```

```
q.emplace(50); // Queue: [10, 20, 30, 40, 50]
```

```
// Push pairs
```

```
queue<pair<int, string>> q2;
```

```
q2.push({1, "Alice"});
```

```
q2.push({2, "Bob"});
```

```
q2.emplace(3, "Charlie"); // Better performance
```

Visual:

Initial: Empty

Push 10:

Front -> [10] <- Back

Push 20:

Front -> [10][20] <- Back

Push 30:

Front -> [10][20][30] <- Back

Push 40:

Front -> [10][20][30][40] <- Back

3 Dequeue (Remove from Front) - O(1)

```
queue<int> q;
```

```
q.push(10);
```

```
q.push(20);
```

```

q.push(30);
q.push(40);
// Queue: [10, 20, 30, 40] (10 is front, 40 is back)

// pop() - removes front element
q.pop(); // Removes 10
// Queue: [20, 30, 40]

cout << "Front after pop: " << q.front(); // 20

q.pop(); // Removes 20
// Queue: [30, 40]

q.pop(); // Removes 30
// Queue: [40]

q.pop(); // Removes 40
// Queue: [] (empty)

// ⚠ IMPORTANT: pop() does NOT return the value
// ❌ int x = q.pop(); // ERROR!

// ✅ Correct way to get and remove:
int x = q.front(); // Get value
q.pop();           // Then remove

// Always check before pop
if(!q.empty()) {
    q.pop();
}

```

Visual:

Before:

Front -> [10][20][30][40] <- Back

After `pop()`:
Front -> [20][30][40] <- Back

After `pop()`:
Front -> [30][40] <- Back

After `pop()`:
Front -> [40] <- Back

After `pop()`:
Empty

4 Access Elements - O(1)

```
queue<int> q;  
q.push(10);  
q.push(20);  
q.push(30);  
q.push(40);  
  
// front() - returns reference to front element  
cout << q.front(); // 10 (first inserted)  
  
// back() - returns reference to back element  
cout << q.back(); // 40 (last inserted)  
  
// Modify front element  
q.front() = 99;  
cout << q.front(); // 99  
  
// Modify back element
```

```

q.back() = 88;
cout << q.back();    // 88

// ❌ q[0] - NOT POSSIBLE (no random access)
// ❌ q.at(2) - NOT POSSIBLE

// front() and back() on empty queue is UNDEFINED BEHAVIOR
queue<int> empty_q;
// cout << empty_q.front(); // ❌ DANGEROUS!

// Safe access
if(!q.empty()) {
    cout << q.front(); // Safe
    cout << q.back();  // Safe
}

```

Example:

```

queue<string> customers;
customers.push("Alice");
customers.push("Bob");
customers.push("Charlie");

cout << "Next customer: " << customers.front() << endl; // Alice
cout << "Last customer: " << customers.back() << endl;  //
Charlie

customers.pop(); // Alice served
cout << "Next customer: " << customers.front() << endl; // Bob

```

5 Size & Empty Check - O(1)

```

queue<int> q;

// empty() - returns true if queue is empty
cout << q.empty(); // 1 (true)

q.push(10);
q.push(20);
q.push(30);

// size() - returns number of elements
cout << q.size(); // 3
cout << q.empty(); // 0 (false)

q.pop();
cout << q.size(); // 2

q.pop();
q.pop();
cout << q.empty(); // 1 (true)
cout << q.size(); // 0

```

Example:

```

queue<int> q;
q.push(10);
q.push(20);
q.push(30);

cout << "Queue size: " << q.size() << endl; // 3

// Process all elements
while(!q.empty()) {
    cout << "Processing: " << q.front() << endl;
    q.pop();
}

```

```
}  
// Output:  
// Processing: 10  
// Processing: 20  
// Processing: 30  
  
cout << "Queue size after: " << q.size() << endl;  // 0
```

6 Traversal (Display All Elements)

Important: Queue doesn't support direct iteration. To traverse, you need to pop elements (which destroys the queue).

```
queue<int> q;  
q.push(10);  
q.push(20);  
q.push(30);  
q.push(40);  
  
// Method 1: Pop all elements (destroys queue)  
cout << "Queue elements: ";  
while(!q.empty()) {  
    cout << q.front() << " ";  
    q.pop();  
}  
  
// Output: 10 20 30 40  
// Queue is now empty  
  
// Method 2: Copy to temporary queue (preserves original)  
void displayQueue(queue<int> q) { // Pass by value (creates  
copy)  
    while(!q.empty()) {
```



```

        cout << q.front() << " ";
        q.pop();
    }
    cout << endl;
}

queue<int> original;
original.push(10);
original.push(20);
original.push(30);

displayQueue(original);    // 10 20 30
cout << original.size();    // 3 (original preserved)

// Method 3: Copy to vector for traversal
queue<int> q2;
q2.push(10);
q2.push(20);
q2.push(30);

vector<int> v;
queue<int> temp = q2;    // Copy

while(!temp.empty()) {
    v.push_back(temp.front());
    temp.pop();
}

// Display from vector
for(int x : v) {
    cout << x << " ";    // 10 20 30
}

// Original queue preserved
cout << "\nOriginal size: " << q2.size();    // 3

```

7 Swap Queues - O(1)

```
queue<int> q1;
q1.push(10);
q1.push(20);

queue<int> q2;
q2.push(30);
q2.push(40);
q2.push(50);

cout << "q1 size: " << q1.size(); // 2
cout << "q2 size: " << q2.size(); // 3

// swap() - exchanges contents
q1.swap(q2);

cout << "After swap:" << endl;
cout << "q1 size: " << q1.size(); // 3
cout << "q2 size: " << q2.size(); // 2

cout << "q1 front: " << q1.front(); // 30
cout << "q2 front: " << q2.front(); // 10
```



Time Complexity Summary (Queue)

Operation	Time Complexity	Space
push()	O(1)	O(1)
pop()	O(1)	O(1)



Operation	Time Complexity	Space
front()	O(1)	O(1)
back()	O(1)	O(1)
size()	O(1)	O(1)
empty()	O(1)	O(1)
swap()	O(1)	O(1)

Space Complexity: O(n) where n = number of elements

Interview Problems with Queue

Problem 1: Reverse a Queue

```
void reverseQueue(queue<int>& q) {
    stack<int> st;

    // Transfer all elements to stack
    while(!q.empty()) {
        st.push(q.front());
        q.pop();
    }

    // Transfer back to queue (reversed)
    while(!st.empty()) {
        q.push(st.top());
        st.pop();
    }
}

queue<int> q;
```

```

q.push(10);
q.push(20);
q.push(30);
q.push(40);

reverseQueue(q);

// Display
while(!q.empty()) {
    cout << q.front() << " ";
    q.pop();
}
// Output: 40 30 20 10

```

How it works:

```

Original Queue: [10, 20, 30, 40]
                ↓
Stack (LIFO):   [10, 20, 30, 40] (40 on top)
                ↓
Reversed Queue: [40, 30, 20, 10]

```

Problem 2: First Non-Repeating Character in Stream

```

void firstNonRepeating(string stream) {
    queue<char> q;
    int freq[26] = {0};

    for(char c : stream) {
        // Increase frequency
        freq[c - 'a']++;
        q.push(c);
    }
}

```

```

        // Remove characters with frequency > 1 from front
        while(!q.empty() && freq[q.front() - 'a'] > 1) {
            q.pop();
        }

        // Print first non-repeating or -1
        if(q.empty()) {
            cout << "-1 ";
        } else {
            cout << q.front() << " ";
        }
    }
}

firstNonRepeating("aabccb");
// Output: a -1 b b b -1

```

Explanation:

Stream: a a b c c b

After 'a': Queue=[a], Output=a

After 'a': Queue=[], Output=-1 (a repeated)

After 'b': Queue=[b], Output=b

After 'c': Queue=[b,c], Output=b

After 'c': Queue=[b], Output=b (c repeated)

After 'b': Queue=[], Output=-1 (b repeated)

Problem 3: Generate Binary Numbers from 1 to N

```

vector<string> generateBinary(int n) {
    vector<string> result;
    queue<string> q;

    q.push("1");

    for(int i = 0; i < n; i++) {
        string front = q.front();
        q.pop();

        result.push_back(front);

        q.push(front + "0");
        q.push(front + "1");
    }

    return result;
}

vector<string> binary = generateBinary(8);
for(string s : binary) {
    cout << s << " ";
}

// Output: 1 10 11 100 101 110 111 1000

```

How it works:

Start: Queue = [1]

Pop "1", add to result

Push "10", "11": Queue = [10, 11]

Pop "10", add to result

Push "100", "101": Queue = [11, 100, 101]

```
Pop "11", add to result
Push "110", "111": Queue = [100, 101, 110, 111]

... continues
```

Problem 4: Interleave First Half with Second Half

```
void interleave(queue<int>& q) {
    int n = q.size();
    queue<int> firstHalf;

    // Get first half
    for(int i = 0; i < n/2; i++) {
        firstHalf.push(q.front());
        q.pop();
    }

    // Interleave
    while(!firstHalf.empty()) {
        q.push(firstHalf.front());
        firstHalf.pop();

        q.push(q.front());
        q.pop();
    }
}

queue<int> q;
for(int i = 1; i <= 8; i++) {
    q.push(i);
}
```

```
// Queue: [1, 2, 3, 4, 5, 6, 7, 8]

interleave(q);

while(!q.empty()) {
    cout << q.front() << " ";
    q.pop();
}

// Output: 1 5 2 6 3 7 4 8
```

Explanation:

Original: [1, 2, 3, 4, 5, 6, 7, 8]

First Half: [1, 2, 3, 4]

Second Half: [5, 6, 7, 8]

Interleaved: [1, 5, 2, 6, 3, 7, 4, 8]

Problem 5: Implement Stack Using Two Queues

```
class StackUsingQueues {
    queue<int> q1, q2;

public:
    // Push - O(n)
    void push(int x) {
        q2.push(x);

        while(!q1.empty()) {
            q2.push(q1.front());
            q1.pop();
        }
    }
};
```



```

    }

    swap(q1, q2);
}

// Pop - O(1)
void pop() {
    if(!q1.empty()) {
        q1.pop();
    }
}

// Top - O(1)
int top() {
    return q1.front();
}

// Empty - O(1)
bool empty() {
    return q1.empty();
}
};

// Usage
StackUsingQueues st;
st.push(10);
st.push(20);
st.push(30);

cout << st.top(); // 30 (LIFO)
st.pop();
cout << st.top(); // 20

```

How push works:



```
Push 10:
q2 = [10]
q1 = []
After swap: q1 = [10]

Push 20:
q2 = [20]
Move q1 to q2: q2 = [20, 10]
After swap: q1 = [20, 10]

Push 30:
q2 = [30]
Move q1 to q2: q2 = [30, 20, 10]
After swap: q1 = [30, 20, 10]

Now q1.front() = 30 (acts like stack top)
```

Problem 6: Reverse First K Elements of Queue

```
void reverseK(queue<int>& q, int k) {
    if(k <= 0 || k > q.size()) return;

    stack<int> st;

    // Push first k elements to stack
    for(int i = 0; i < k; i++) {
        st.push(q.front());
        q.pop();
    }

    // Push back from stack (reversed)
    while(!st.empty()) {
```

```

        q.push(st.top());
        st.pop();
    }

    // Move remaining n-k elements to back
    int remaining = q.size() - k;
    for(int i = 0; i < remaining; i++) {
        q.push(q.front());
        q.pop();
    }
}

queue<int> q;
for(int i = 10; i <= 60; i += 10) {
    q.push(i);
}
// Queue: [10, 20, 30, 40, 50, 60]

reverseK(q, 3);

while(!q.empty()) {
    cout << q.front() << " ";
    q.pop();
}
// Output: 30 20 10 40 50 60

```

Steps:

Original: [10, 20, 30, 40, 50, 60]

First 3 to stack: [10, 20, 30]

Queue: [40, 50, 60]

Pop from stack: Queue: [40, 50, 60, 30, 20, 10]

Move $(6-3)=3$ elements to back:

[40, 50, 60, 30, 20, 10]

↓

[30, 20, 10, 40, 50, 60]

Problem 7: Find Maximum in All Subarrays of Size K (Sliding Window)

```
vector<int> maxSlidingWindow(vector<int>& arr, int k) {
    vector<int> result;
    deque<int> dq; // Store indices

    for(int i = 0; i < arr.size(); i++) {
        // Remove elements outside current window
        if(!dq.empty() && dq.front() == i - k) {
            dq.pop_front();
        }

        // Remove smaller elements from back
        while(!dq.empty() && arr[dq.back()] < arr[i]) {
            dq.pop_back();
        }

        dq.push_back(i);

        // Add to result after first window
        if(i >= k - 1) {
            result.push_back(arr[dq.front()]);
        }
    }

    return result;
}
```

```

}

vector<int> arr = {1, 3, -1, -3, 5, 3, 6, 7};
vector<int> result = maxSlidingWindow(arr, 3);

for(int x : result) {
    cout << x << " ";
}

// Output: 3 3 5 5 6 7

```

Explanation:

Array: [1, 3, -1, -3, 5, 3, 6, 7], k=3

Window [1, 3, -1]: max = 3

Window [3, -1, -3]: max = 3

Window [-1, -3, 5]: max = 5

Window [-3, 5, 3]: max = 5

Window [5, 3, 6]: max = 6

Window [3, 6, 7]: max = 7

Problem 8: Circular Tour (Gas Station)

```

int canCompleteCircuit(vector<int>& gas, vector<int>& cost) {
    int n = gas.size();
    int totalGas = 0, totalCost = 0;
    int tank = 0, start = 0;

    for(int i = 0; i < n; i++) {
        totalGas += gas[i];
        totalCost += cost[i];
    }
}

```

```

        tank += gas[i] - cost[i];

        if(tank < 0) {
            start = i + 1;
            tank = 0;
        }
    }

    return (totalGas >= totalCost) ? start : -1;
}

vector<int> gas = {1, 2, 3, 4, 5};
vector<int> cost = {3, 4, 5, 1, 2};

cout << canCompleteCircuit(gas, cost); // 3
// Start from station 3 to complete the tour

```

Problem 9: Time Needed to Buy Tickets

```

int timeRequiredToBuy(vector<int>& tickets, int k) {
    int time = 0;
    queue<pair<int, int>> q; // {tickets_count, index}

    for(int i = 0; i < tickets.size(); i++) {
        q.push({tickets[i], i});
    }

    while(!q.empty()) {
        auto front = q.front();
        q.pop();

        time++;
    }
}

```

```

        front.first--;

        if(front.second == k && front.first == 0) {
            return time;
        }

        if(front.first > 0) {
            q.push(front);
        }
    }

    return time;
}

vector<int> tickets = {2, 3, 2};
cout << timeRequiredToBuy(tickets, 2); // 6
// Person at index 2 needs 6 seconds to buy tickets

```

Problem 10: Level Order Traversal (BFS on Tree)

```

struct TreeNode {
    int val;
    TreeNode* left;
    TreeNode* right;
    TreeNode(int x) : val(x), left(NULL), right(NULL) {}
};

vector<vector<int>> levelOrder(TreeNode* root) {
    vector<vector<int>> result;
    if(root == NULL) return result;

    queue<TreeNode*> q;

```

```

q.push(root);

while(!q.empty()) {
    int size = q.size();
    vector<int> level;

    for(int i = 0; i < size; i++) {
        TreeNode* node = q.front();
        q.pop();

        level.push_back(node->val);

        if(node->left) q.push(node->left);
        if(node->right) q.push(node->right);
    }

    result.push_back(level);
}

return result;
}

```

// Example Tree:

```

//      1
//     / \
//    2   3
//   / \
//  4   5

```

```

TreeNode* root = new TreeNode(1);
root->left = new TreeNode(2);
root->right = new TreeNode(3);
root->left->left = new TreeNode(4);
root->left->right = new TreeNode(5);

```



```
vector<vector<int>> levels = levelOrder(root);  
// Output: [[1], [2,3], [4,5]]
```

⚠ Common Mistakes & Tips (Queue)

❌ Mistake 1: Using front() on empty queue

```
queue<int> q;  
cout << q.front(); // ❌ Undefined behavior
```

✅ Solution:

```
if(!q.empty()) {  
    cout << q.front(); // Safe  
}
```

❌ Mistake 2: Thinking pop() returns value

```
queue<int> q;  
q.push(10);  
int x = q.pop(); // ❌ ERROR! pop() returns void
```

✅ Solution:

```
int x = q.front(); // Get value first  
q.pop();           // Then remove
```

❌ Mistake 3: Trying random access

```
queue<int> q;  
q.push(10);  
q.push(20);  
cout << q[1]; // ❌ NOT POSSIBLE
```

✅ Solution:

```
// Queue doesn't support random access  
// Only front() and back() available  
cout << q.front(); // First element  
cout << q.back();  // Last element
```

Shall I continue with **DEQUE (Double Ended Queue)** next? 🚀

08/01/2026, 12:38:44

HASH MAP (unordered_map)

What is Hash Map?

Hash Map (unordered_map) ek data structure hai jo **key-value pairs** store karta hai. Har key unique hoti hai aur use karke corresponding value ko **O(1) average time** me access kar sakte ho.

Real Life Examples:

- **Phone book** - Name (key) → Phone Number (value)
- **Dictionary** - Word (key) → Meaning (value)
- **Student records** - Roll Number (key) → Student Details (value)
- **Shopping cart** - Product ID (key) → Quantity (value)
- **Cache** - URL (key) → Cached Data (value)

Visual:

Key	→	Value

"apple"	→	5
"banana"	→	3
"orange"	→	8
"mango"	→	2

How Hash Map Works Internally?

Hashing Process:

```
Key: "apple"
  ↓
Hash Function
  ↓
Hash Code: 12345
  ↓
Index: 12345 % bucket_size = 5
  ↓
Store at Bucket[5]: {"apple": 5}
```

Internal Structure:

```
Buckets (Array of Linked Lists):

Bucket 0: NULL
Bucket 1: {"banana": 3} → NULL
Bucket 2: NULL
Bucket 3: {"mango": 2} → NULL
Bucket 4: NULL
Bucket 5: {"apple": 5} → {"orange": 8} → NULL (collision handled)
Bucket 6: NULL
...
```

Key Points:

- Uses **hash function** to convert key to index
- Handles **collisions** using chaining (linked lists)
- **Unordered** - no particular order of keys
- Average **O(1)** access time
- Worst case **O(n)** if many collisions



Complete unordered_map<> Operations

1 Declaration & Initialization

```
#include <unordered_map>
#include <iostream>
using namespace std;

// Method 1: Empty map
unordered_map<int, int> m;

// Method 2: Initialize with values
unordered_map<int, int> m = {{1, 100}, {2, 200}, {3, 300}};

// Method 3: Different key-value types
unordered_map<string, int> m; // string → int
unordered_map<int, string> m; // int → string
unordered_map<char, int> m; // char → int
unordered_map<string, double> m; // string → double

// Method 4: Complex value types
unordered_map<int, vector<int>> m; // int → vector
unordered_map<string, pair<int, int>> m; // string → pair

// Method 5: Copy from another map
unordered_map<int, int> m1 = {{1, 10}, {2, 20}};
unordered_map<int, int> m2 = m1; // Copy
unordered_map<int, int> m3(m1); // Copy constructor

// Method 6: Using initializer list
unordered_map<string, int> fruits = {
    {"apple", 5},
    {"banana", 3},
    {"orange", 8}
};
```



```
// Method 7: Pair type
unordered_map<pair<int, int>, string> m; // ❌ NOT ALLOWED (pair
as key needs hash function)
```

Example:

```
#include <iostream>
#include <unordered_map>
using namespace std;

int main() {
    unordered_map<string, int> ages;

    ages["Alice"] = 25;
    ages["Bob"] = 30;
    ages["Charlie"] = 28;

    cout << "Alice's age: " << ages["Alice"] << endl; // 25
    cout << "Map size: " << ages.size() << endl;      // 3

    return 0;
}
```

2 Insertion - O(1) average

```
unordered_map<int, int> m;

// Method 1: Using [] operator
m[1] = 100; // Insert key=1, value=100
m[2] = 200;
m[3] = 300;
```

```

// If key exists, it updates the value
m[1] = 150; // Updates value of key 1 to 150

// Method 2: Using insert()
m.insert({4, 400});
m.insert(make_pair(5, 500));
m.insert(pair<int, int>(6, 600));

// insert() returns pair<iterator, bool>
auto result = m.insert({7, 700});
if(result.second) {
    cout << "Inserted successfully";
} else {
    cout << "Key already exists";
}

// Insert only if key doesn't exist
m.insert({1, 999}); // Won't insert (key 1 exists)
cout << m[1]; // Still 150

// Method 3: Using emplace() (faster, constructs in place)
m.emplace(8, 800);

// Method 4: String keys
unordered_map<string, int> fruits;
fruits["apple"] = 5;
fruits["banana"] = 3;
fruits.insert({"orange", 8});

// Method 5: Insert multiple at once
unordered_map<int, int> m2 = {{10, 1000}, {20, 2000}};

```

Example:

```
unordered_map<string, int> marks;

// Adding student marks
marks["Rahul"] = 85;
marks["Priya"] = 92;
marks["Amit"] = 78;

// Update mark
marks["Rahul"] = 88; // Updated from 85 to 88

// Display
for(auto p : marks) {
    cout << p.first << ": " << p.second << endl;
}
// Output (unordered):
// Amit: 78
// Priya: 92
// Rahul: 88
```

3 Access - O(1) average

```
unordered_map<int, int> m = {{1, 100}, {2, 200}, {3, 300}};

// Method 1: Using [] operator
cout << m[1]; // 100
cout << m[2]; // 200

// ⚠ If key doesn't exist, [] creates it with default value (0
for int)
cout << m[10]; // 0 (key 10 created automatically)
cout << m.size(); // 4 (now includes key 10)
```



```

// Method 2: Using at() (safer - throws exception if key not
found)
cout << m.at(1); // 100

try {
    cout << m.at(100); // ✗ Throws out_of_range exception
} catch(exception &e) {
    cout << "Key not found: " << e.what();
}

// Method 3: Check existence before access
if(m.find(2) != m.end()) {
    cout << "Key 2 exists: " << m[2];
}

// Method 4: Using count() to check existence
if(m.count(3)) {
    cout << "Key 3 exists: " << m[3];
}

// String keys
unordered_map<string, int> ages = {"Alice", 25}, {"Bob", 30}};
cout << ages["Alice"]; // 25
cout << ages["Bob"]; // 30

```

Example:

```

unordered_map<string, string> phoneBook;
phoneBook["Alice"] = "9876543210";
phoneBook["Bob"] = "9123456789";

// Safe access
string name = "Alice";
if(phoneBook.count(name)) {
    cout << name << "'s phone: " << phoneBook[name];
}

```

```

} else {
    cout << "Contact not found";
}

// Output: Alice's phone: 9876543210

// Unsafe access
cout << phoneBook["Charlie"]; // Creates key "Charlie" with
empty string

```

4 Update - O(1) average

```

unordered_map<int, int> m = {{1, 100}, {2, 200}, {3, 300}};

// Update using []
m[1] = 150; // 100 → 150
m[2] = 250; // 200 → 250

// Update using at()
m.at(3) = 350; // 300 → 350

// Update using iterator
auto it = m.find(1);
if(it != m.end()) {
    it->second = 999; // Update value
}

cout << m[1]; // 999

// Update all values
for(auto &p : m) {
    p.second = p.second * 2; // Double all values
}

```

Example:

```
unordered_map<string, int> inventory;
inventory["apples"] = 50;
inventory["bananas"] = 30;

// Sell 10 apples
inventory["apples"] -= 10;
cout << "Apples remaining: " << inventory["apples"]; // 40

// Add 20 bananas
inventory["bananas"] += 20;
cout << "Bananas remaining: " << inventory["bananas"]; // 50
```

5 Deletion - O(1) average

```
unordered_map<int, int> m = {{1, 100}, {2, 200}, {3, 300}, {4, 400}};

// Method 1: erase(key) - by key
m.erase(1); // Removes key 1
cout << m.size(); // 3

// Check if erased successfully
if(m.erase(2)) {
    cout << "Key 2 removed";
} else {
    cout << "Key 2 not found";
}

// erase() non-existent key - no error, returns 0
int removed = m.erase(100); // Returns 0 (not found)
```

```

// Method 2: erase(iterator) - by iterator
auto it = m.find(3);
if(it != m.end()) {
    m.erase(it); // Removes key 3
}

// Method 3: erase(start, end) - range erase
unordered_map<int, int> m2 = {{1,10}, {2,20}, {3,30}, {4,40}};
auto start = m2.find(2);
auto end = m2.find(4);
m2.erase(start, end); // Removes keys 2,3 (not 4)

// Method 4: clear() - remove all
m.clear();
cout << m.size(); // 0
cout << m.empty(); // 1 (true)

```

Example:

```

unordered_map<string, int> cart;
cart["apple"] = 5;
cart["banana"] = 3;
cart["orange"] = 8;

// Remove item from cart
cart.erase("banana");

// Check if item removed
if(cart.count("banana") == 0) {
    cout << "Banana removed from cart";
}

// Empty cart

```

```
cart.clear();  
cout << "Cart size: " << cart.size(); // 0
```

6 Search & Check Existence - $O(1)$ average

```
unordered_map<int, int> m = {{1, 100}, {2, 200}, {3, 300}};  
  
// Method 1: Using find() - returns iterator  
auto it = m.find(2);  
if(it != m.end()) {  
    cout << "Key 2 found, value: " << it->second; // 200  
} else {  
    cout << "Key 2 not found";  
}  
  
// Find non-existent key  
auto it2 = m.find(100);  
if(it2 == m.end()) {  
    cout << "Key 100 not found";  
}  
  
// Method 2: Using count() - returns 0 or 1  
if(m.count(3)) {  
    cout << "Key 3 exists"; // Exists  
}  
  
if(m.count(100) == 0) {  
    cout << "Key 100 doesn't exist";  
}  
  
// Method 3: Using [] (but creates key if not exists)  
// ❌ Not recommended for checking existence
```

```
// Method 4: Using at() with try-catch
try {
    int value = m.at(2);
    cout << "Key 2 exists, value: " << value;
} catch(out_of_range &e) {
    cout << "Key doesn't exist";
}
```

Example:

```
unordered_map<string, int> scores;
scores["Alice"] = 95;
scores["Bob"] = 88;
scores["Charlie"] = 92;

// Check if student exists
string name = "Alice";
if(scores.find(name) != scores.end()) {
    cout << name << " scored " << scores[name];
} else {
    cout << name << " not found";
}
// Output: Alice scored 95

// Check multiple students
vector<string> students = {"Alice", "David", "Bob"};
for(string s : students) {
    if(scores.count(s)) {
        cout << s << " found\n";
    } else {
        cout << s << " not found\n";
    }
}
// Output:
```

```
// Alice found
// David not found
// Bob found
```

7 Traversal (Iterate) - O(n)

```
unordered_map<int, int> m = {{1, 100}, {2, 200}, {3, 300}};
```

```
// Method 1: Using range-based for loop (C++11)
```

```
for(auto p : m) {
    cout << p.first << " -> " << p.second << endl;
}
```

```
// Output (unordered):
```

```
// 3 -> 300
```

```
// 1 -> 100
```

```
// 2 -> 200
```

```
// Method 2: Using structured binding (C++17)
```

```
for(auto [key, value] : m) {
    cout << key << " -> " << value << endl;
}
```

```
// Method 3: Using reference (for modification)
```

```
for(auto &p : m) {
    p.second = p.second * 2; // Double all values
}
```

```
// Method 4: Using iterators
```

```
for(auto it = m.begin(); it != m.end(); it++) {
    cout << it->first << " -> " << it->second << endl;
}
```

```

// Method 5: Print only keys
for(auto p : m) {
    cout << p.first << " ";
}

// Method 6: Print only values
for(auto p : m) {
    cout << p.second << " ";
}

// String keys
unordered_map<string, int> fruits = {"apple", 5}, {"banana",
3}};
for(auto [fruit, count] : fruits) {
    cout << fruit << ": " << count << endl;
}

```

Example:

```

unordered_map<string, vector<int>> studentMarks;
studentMarks["Alice"] = {85, 90, 88};
studentMarks["Bob"] = {78, 82, 80};

// Print all student marks
for(auto [name, marks] : studentMarks) {
    cout << name << ": ";
    for(int mark : marks) {
        cout << mark << " ";
    }
    cout << endl;
}

// Output:
// Bob: 78 82 80
// Alice: 85 90 88

```


8 Size & Empty Check - O(1)

```
unordered_map<int, int> m;

// empty() - check if empty
cout << m.empty(); // 1 (true)

m[1] = 100;
m[2] = 200;
m[3] = 300;

// size() - number of key-value pairs
cout << m.size(); // 3
cout << m.empty(); // 0 (false)

m.erase(2);
cout << m.size(); // 2

m.clear();
cout << m.size(); // 0
cout << m.empty(); // 1 (true)

// max_size() - maximum possible size
cout << m.max_size(); // Very large number
```

9 Other Useful Operations

a) Swap Maps - O(1)

```
unordered_map<int, int> m1 = {{1, 10}, {2, 20}};
unordered_map<int, int> m2 = {{3, 30}, {4, 40}, {5, 50}};
```

```
cout << "m1 size: " << m1.size(); // 2
cout << "m2 size: " << m2.size(); // 3

m1.swap(m2);

cout << "After swap:";
cout << "m1 size: " << m1.size(); // 3
cout << "m2 size: " << m2.size(); // 2
```

b) Get All Keys

```
unordered_map<string, int> m = {{"apple", 5}, {"banana", 3},
{"orange", 8}};

vector<string> keys;
for(auto p : m) {
    keys.push_back(p.first);
}

// Display keys
for(string k : keys) {
    cout << k << " ";
}

// Output: orange apple banana (unordered)
```

c) Get All Values

```
unordered_map<string, int> m = {{"apple", 5}, {"banana", 3},
{"orange", 8}};
```

```
vector<int> values;
for(auto p : m) {
    values.push_back(p.second);
}

// Display values
for(int v : values) {
    cout << v << " ";
}
// Output: 8 5 3 (unordered)
```

d) Bucket Operations

```
unordered_map<int, int> m = {{1, 10}, {2, 20}, {3, 30}};

// bucket_count() - number of buckets
cout << "Buckets: " << m.bucket_count() << endl;

// load_factor() - average elements per bucket
cout << "Load factor: " << m.load_factor() << endl;

// max_load_factor() - maximum load factor
cout << "Max load factor: " << m.max_load_factor() << endl;

// bucket(key) - which bucket contains key
cout << "Key 1 in bucket: " << m.bucket(1) << endl;

// bucket_size(bucket_index) - elements in bucket
cout << "Bucket 0 size: " << m.bucket_size(0) << endl;
```



Time Complexity Summary

Operation	Average	Worst Case	Explanation
Insert	$O(1)$	$O(n)$	Worst when rehashing needed
Delete	$O(1)$	$O(n)$	Worst with many collisions
Search/Access	$O(1)$	$O(n)$	Worst with many collisions
Traversal	$O(n)$	$O(n)$	Visit all elements
Size/Empty	$O(1)$	$O(1)$	Direct counter

Space Complexity: $O(n)$ where n = number of key-value pairs



Interview Problems with unordered_map

Problem 1: Frequency Count

```
unordered_map<int, int> frequencyCount(vector<int> arr) {
    unordered_map<int, int> freq;

    for(int x : arr) {
        freq[x]++;
    }

    return freq;
}

vector<int> arr = {1, 2, 2, 3, 3, 3, 4, 4, 4, 4};
auto freq = frequencyCount(arr);

for(auto [num, count] : freq) {
    cout << num << " appears " << count << " times\n";
}
```



```
}  
// Output:  
// 4 appears 4 times  
// 3 appears 3 times  
// 2 appears 2 times  
// 1 appears 1 times
```

Problem 2: Two Sum

```
vector<int> twoSum(vector<int>& nums, int target) {  
    unordered_map<int, int> m; // value -> index  
  
    for(int i = 0; i < nums.size(); i++) {  
        int complement = target - nums[i];  
  
        if(m.find(complement) != m.end()) {  
            return {m[complement], i};  
        }  
  
        m[nums[i]] = i;  
    }  
  
    return {}; // Not found  
}  
  
vector<int> nums = {2, 7, 11, 15};  
int target = 9;  
  
vector<int> result = twoSum(nums, target);  
cout << "Indices: " << result[0] << ", " << result[1];  
// Output: Indices: 0, 1 (2+7=9)
```

How it works:

Array: [2, 7, 11, 15], target = 9

i=0: nums[0]=2, complement=7

Map: {2: 0}

i=1: nums[1]=7, complement=2

2 found in map at index 0

Return [0, 1]

Problem 3: First Non-Repeating Character

```
char firstNonRepeating(string s) {
    unordered_map<char, int> freq;

    // Count frequency
    for(char c : s) {
        freq[c]++;
    }

    // Find first non-repeating
    for(char c : s) {
        if(freq[c] == 1) {
            return c;
        }
    }

    return '\0'; // None found
}

cout << firstNonRepeating("leetcode"); // 'l'
```

```
cout << firstNonRepeating("loveleetcode"); // 'v'
cout << firstNonRepeating("aabb");           // '\0'
```

Problem 4: Group Anagrams

```
vector<vector<string>> groupAnagrams(vector<string>& strs) {
    unordered_map<string, vector<string>> m;

    for(string s : strs) {
        string key = s;
        sort(key.begin(), key.end());
        m[key].push_back(s);
    }

    vector<vector<string>> result;
    for(auto [key, group] : m) {
        result.push_back(group);
    }

    return result;
}

vector<string> words = {"eat", "tea", "tan", "ate", "nat",
                       "bat"};
auto groups = groupAnagrams(words);

for(auto group : groups) {
    for(string word : group) {
        cout << word << " ";
    }
    cout << endl;
}
```

```
// Output:  
// bat  
// tan nat  
// eat tea ate
```

Problem 5: Longest Consecutive Sequence

```
int longestConsecutive(vector<int>& nums) {  
    unordered_map<int, bool> m;  
  
    // Mark all numbers as present  
    for(int num : nums) {  
        m[num] = true;  
    }  
  
    int maxLength = 0;  
  
    for(int num : nums) {  
        // If num-1 doesn't exist, num is start of sequence  
        if(m.find(num - 1) == m.end()) {  
            int currentNum = num;  
            int currentLength = 1;  
  
            // Count consecutive numbers  
            while(m.find(currentNum + 1) != m.end()) {  
                currentNum++;  
                currentLength++;  
            }  
  
            maxLength = max(maxLength, currentLength);  
        }  
    }  
}
```



```
        return maxLength;
    }

    vector<int> nums = {100, 4, 200, 1, 3, 2};
    cout << longestConsecutive(nums); // 4 (1,2,3,4)
```

Problem 6: Subarray Sum Equals K

```
int subarraySum(vector<int>& nums, int k) {
    unordered_map<int, int> prefixSum; // sum -> count
    prefixSum[0] = 1;

    int sum = 0, count = 0;

    for(int num : nums) {
        sum += num;

        if(prefixSum.find(sum - k) != prefixSum.end()) {
            count += prefixSum[sum - k];
        }

        prefixSum[sum]++;
    }

    return count;
}

vector<int> nums = {1, 1, 1};
cout << subarraySum(nums, 2); // 2 (subarrays: [1,1] and [1,1])
```

Problem 7: Check if Array Pairs are Divisible by K

```
bool canArrange(vector<int>& arr, int k) {
    unordered_map<int, int> remainderCount;

    for(int num : arr) {
        int remainder = ((num % k) + k) % k;
        remainderCount[remainder]++;
    }

    for(int num : arr) {
        int remainder = ((num % k) + k) % k;

        if(remainder == 0) {
            if(remainderCount[remainder] % 2 != 0) {
                return false;
            }
        } else {
            int complement = k - remainder;
            if(remainderCount[remainder] !=
remainderCount[complement]) {
                return false;
            }
        }
    }

    return true;
}

vector<int> arr = {1, 2, 3, 4, 5, 10, 6, 7, 8, 9};
cout << canArrange(arr, 5); // true
```

Problem 8: Word Pattern

```
bool wordPattern(string pattern, string s) {
    unordered_map<char, string> charToWord;
    unordered_map<string, char> wordToChar;

    vector<string> words;
    stringstream ss(s);
    string word;

    while(ss >> word) {
        words.push_back(word);
    }

    if(pattern.length() != words.size()) {
        return false;
    }

    for(int i = 0; i < pattern.length(); i++) {
        char c = pattern[i];
        string w = words[i];

        if(charToWord.count(c) && charToWord[c] != w) {
            return false;
        }
        if(wordToChar.count(w) && wordToChar[w] != c) {
            return false;
        }

        charToWord[c] = w;
        wordToChar[w] = c;
    }

    return true;
}
```

```
cout << wordPattern("abba", "dog cat cat dog"); // true
cout << wordPattern("abba", "dog cat cat fish"); // false
```

Problem 9: Isomorphic Strings

```
bool isIsomorphic(string s, string t) {
    if(s.length() != t.length()) return false;

    unordered_map<char, char> mapS, mapT;

    for(int i = 0; i < s.length(); i++) {
        char c1 = s[i], c2 = t[i];

        if(mapS.count(c1) && mapS[c1] != c2) {
            return false;
        }
        if(mapT.count(c2) && mapT[c2] != c1) {
            return false;
        }

        mapS[c1] = c2;
        mapT[c2] = c1;
    }

    return true;
}

cout << isIsomorphic("egg", "add"); // true
cout << isIsomorphic("foo", "bar"); // false
cout << isIsomorphic("paper", "title"); // true
```

Problem 10: Top K Frequent Elements

```
vector<int> topKFrequent(vector<int>& nums, int k) {
    unordered_map<int, int> freq;

    // Count frequency
    for(int num : nums) {
        freq[num]++;
    }

    // Use max heap (priority queue)
    priority_queue<pair<int, int>> pq;

    for(auto [num, count] : freq) {
        pq.push({count, num});
    }

    // Get top k
    vector<int> result;
    for(int i = 0; i < k; i++) {
        result.push_back(pq.top().second);
        pq.pop();
    }

    return result;
}

vector<int> nums = {1,1,1,2,2,3};
vector<int> result = topKFrequent(nums, 2);

for(int x : result) {
    cout << x << " ";
}
```

```
}  
// Output: 1 2
```

⚠ Common Mistakes & Tips

❌ Mistake 1: Using [] creates key if not exists

```
unordered_map<int, int> m;  
if(m[10] == 0) { // ❌ Creates key 10 with value 0  
    cout << "Key 10 doesn't exist";  
}  
cout << m.size(); // 1 (key created!)
```

✅ Solution:

```
if(m.find(10) == m.end()) { // Doesn't create key  
    cout << "Key 10 doesn't exist";  
}  
// OR  
if(m.count(10) == 0) { // Doesn't create key  
    cout << "Key 10 doesn't exist";  
}
```

❌ Mistake 2: Modifying map while iterating

```
unordered_map<int, int> m = {{1,10}, {2,20}, {3,30}};  
  
for(auto p : m) {
```

```
    if(p.second == 20) {  
        m.erase(p.first); // ❌ Invalidates iterator  
    }  
}
```

✅ Solution:

```
for(auto it = m.begin(); it != m.end(); ) {  
    if(it->second == 20) {  
        it = m.erase(it); // erase returns next iterator  
    } else {  
        it++;  
    }  
}
```

❌ Mistake 3: Using custom objects as keys without hash function

```
struct Point {  
    int x, y;  
};  
  
unordered_map<Point, int> m; // ❌ ERROR! No hash function for Point
```

✅ Solution:

```
// Use map instead (requires < operator)  
map<Point, int> m;
```

```
// OR define custom hash
struct PointHash {
    size_t operator()(const Point& p) const {
        return hash<int>()(p.x) ^ hash<int>()(p.y);
    }
};

unordered_map<Point, int, PointHash> m;
```



Pro Tips

1. Use count() or find() to check existence:

```
if(m.count(key)) { } // Better than m[key]
```

2. Frequency counting pattern:

```
unordered_map
```

08/01/2026, 12:40:45

